

HOW TO USE

이 자료 사용 방법

1. 수업·강의·자습에 그대로 쓰셔도 됩니다. 출처만 남겨 주세요.
2. PDF 는 화면으로 읽는 용도, PPTX 는 발표용입니다. PPTX 는 Pretendard 글꼴이 설치된 컴퓨터에서 가장 똑같이 보입니다.
3. 출처 표기 예: 이애본(드림아이티비즈), 「자료 제목」 , useful-ai.dreamitbiz.com
4. 자료 안의 프롬프트는 복사해 바로 쓸 수 있습니다. [대괄호]만 내 상황으로 바꾸세요.
5. 모든 자료와 영상 강의는 useful-ai.dreamitbiz.com 에서 다시 받을 수 있습니다.

저작권 · 이용 조건

1. 저작권 © 2026 이애본(드림아이티비즈 DreamIT Biz). 2026년 1월 「생성형 AI와 창의력·복합문제능력 향상 과정」 강의안을 공개용으로 다시 묶은 것입니다.
2. 교육 목적의 비영리 이용을 허용합니다 — 출처를 표기해 주세요.
3. 재배포·판매·변형(내용 수정 후 배포)은 허용하지 않습니다. 필요하다면 먼저 문의해 주세요.
4. 자료 안의 외부 서비스 화면 캡처와 상표는 교육 목적 인용이며 각 서비스의 소유입니다. AI 생성 이미지는 생성 도구의 이용 약관을 따릅니다.
5. 문의 aebon@dreamitbiz.com · 카카오톡 ID aebon

클로드코드와 제미나이를 이용한 바이트코딩 포트폴리오 배포

AI와 함께하는 초고속 웹 개발 마스터 클래스
환경 설정부터 GitHub Pages 실전 배포까지

강사

이 애 본 (aebon@hs.ac.kr)
한신대학교 AI.SW대학

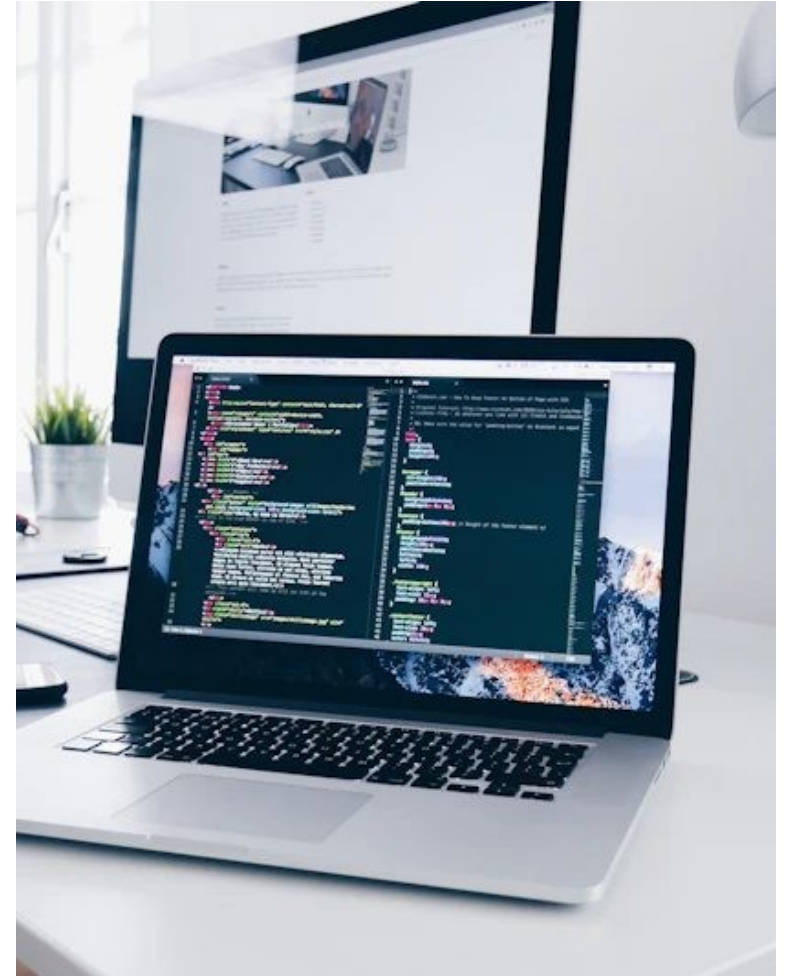
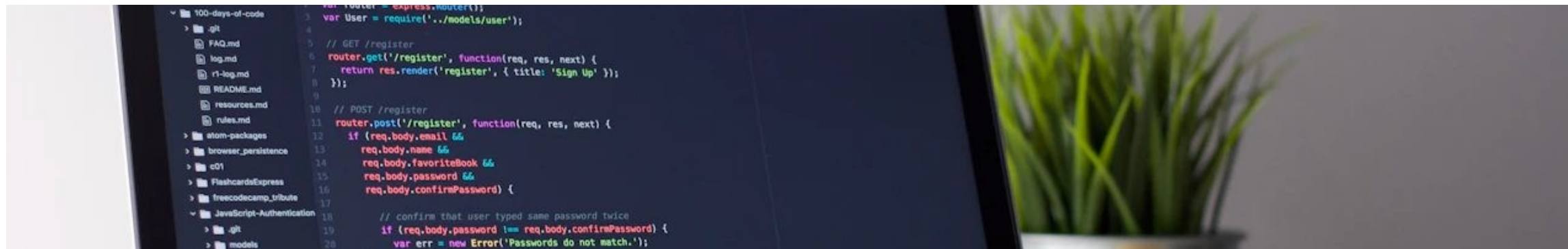


TABLE OF CONTENTS



01 소개 및 준비
바이브코딩 마인드셋과 수업 목표

02 AI 도구 이해
Claude Code & Gemini 활용 전략

03 개발 환경 설정
VS Code, Git, Node.js 셋업

04 바이브코딩 기초
프롬프트 엔지니어링 핵심 기법

05 포트폴리오 구조 설계
HTML/CSS 및 반응형 레이아웃

06 기능 구현 실습
인터랙티브 웹사이트 완성

07 GitHub 배포
Git 명령어와 Pages 호스팅

08 고급 최적화
성능 개선, SEO, 애니메이션

09 마무리
핵심 요약 및 Q&A 세션

01

강의 소개: 왜 바이브코딩인가?

"코딩의 장벽을 AI와 함께 넘어,
아이디어를 즉시 현실로 만듭니다."

바이브코딩(Vibe Coding)은 AI와 개발자가 리듬을 타듯 대화를 주고받으며 결과물을 만들어내는 새로운 개발 방식입니다.

- ✓ 실시간 대화형 개발로 속도와 품질 동시 향상
- ✓ AI와 페어 프로그래밍으로 학습 곡선 단축
- ✓ 목표: 완성도 있는 포트폴리오를 '오늘' 배포

INTRODUCTION



Fast & Efficient

LEARNING OBJECTIVES

01 AI 코딩 도구의 완벽한 이해

Claude Code와 Gemini Code Assist의 장단점과 최적의 사용 시나리오를 파악합니다.

02 실전 프롬프트 작성 기법

AI에게 명확하게 지시하고, 원하는 코드 품질을 얻어내는 바이브코딩 핵심 패턴을 익힙니다.

03 반응형 포트폴리오 제작

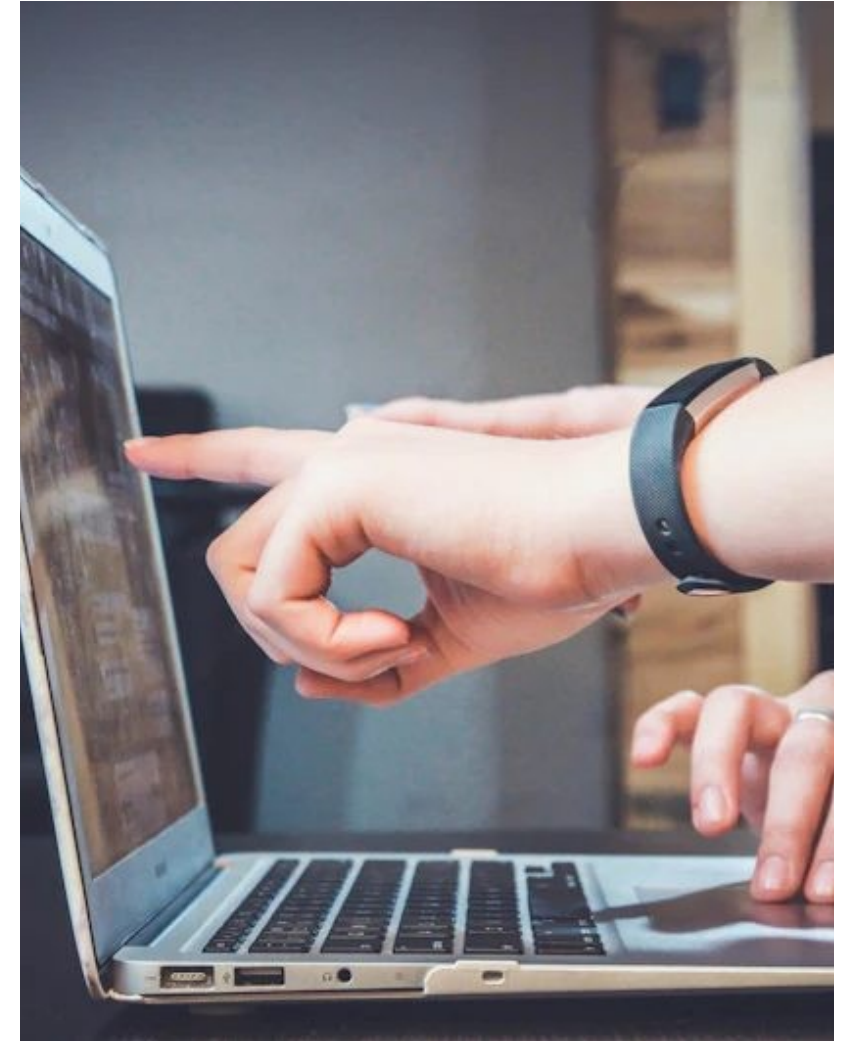
HTML/CSS/JS를 활용하여 모바일과 데스크톱 모두에서 완벽하게 작동하는 웹사이트를 구축합니다.

04 GitHub Pages 배포

Git 기본 명령어를 익히고, 자신의 포트폴리오를 전 세계에 무료로 배포합니다.

05 고급 기능 적용

다크모드, SEO 최적화, 애니메이션 등 완성도를 높이는 고급 기술을 적용합니다.



CLASS PREPARATION

01 필수 소프트웨어 설치

개발을 위해 **VS Code**(코드 에디터), **Git**(버전 관리),
Node.js LTS(자바스크립트 런타임)를 사전에 설치해야 합니다.

02 계정 및 API 준비

배포를 위한 **GitHub** 계정과, AI 코딩을 위한 **Claude.ai** 및 **Google Gemini** 계정 생성이
필요합니다.

03 웹 브라우저

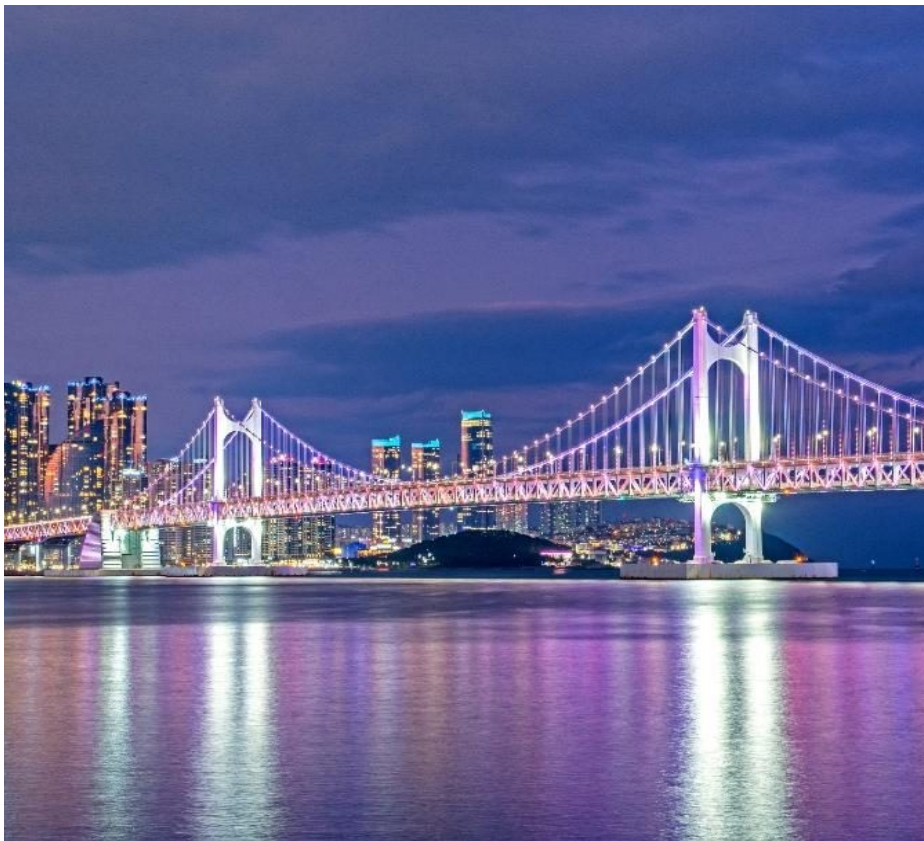
최신 웹 표준 기술과 강력한 개발자 도구(DevTools)를 지원하는
Google Chrome 브라우저를 권장합니다.

04 기본 선수 지식

AI가 코드를 작성해주지만, 구조 이해를 위해 HTML 태그, CSS 선택자,
JavaScript 기초 문법에 대한 지식이 있으면 유리합니다.



3-HOUR ROADMAP



강의 진행 순서

0-20m 소개 및 준비

Intro & Prep

20-50m

AI 도구 이해 (Claude & Gemini)

AI Tools

50-75m

개발 환경 설정 및 체크포인트

Env Setup

80-105m

바이트코드 기초 및 프롬프트 전략

Basics

105-140m

포트폴리오 실전 제작

Portfolio Build

145-175m

배포 및 고급 기능 구현

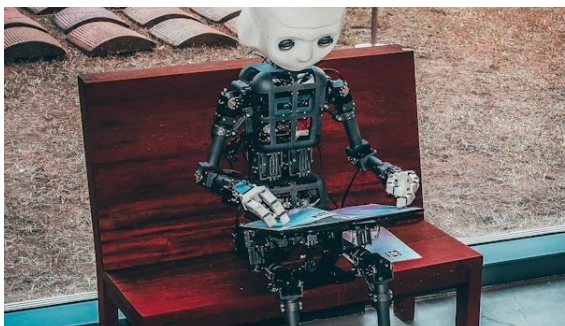
Deployment

175-180m

마무리 및 Q&A

Wrap-up

MEET CLAUDE



대화형 코드 생성, 리뷰, 리팩토링을 돕는 Anthropic의 차세대 AI.
압도적인 컨텍스트 이해력과 안전성을 경험하세요.



CODE

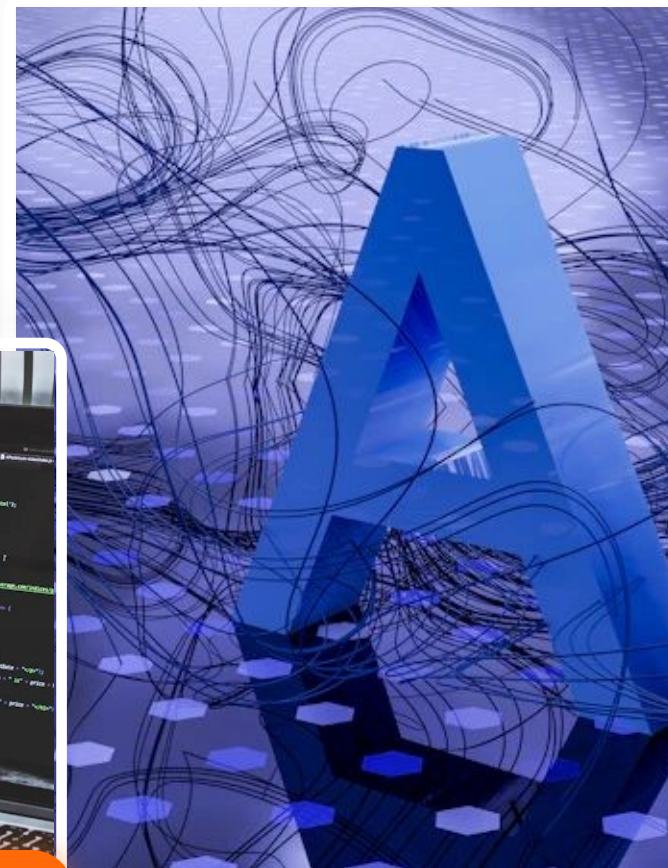
02

Claude란? Anthropic의 차세대 AI

"안전하고 유능하며 신뢰할 수 있는 AI,
긴 문맥 이해와 코딩 보조에 최적화되었습니다."

Claude는 Anthropic이 개발한 대규모 언어 모델(LLM)로, 방대한 컨텍스트 창을 통해 복잡한 프로젝트 코드를 한 번에 이해하고 분석할 수 있는 능력을 갖추고 있습니다.

- ✓ 최대 200K 토큰의 방대한 문맥(Context) 처리
- ✓ Constitutional AI 기술로 안전성과 신뢰성 강화
- ✓ 코드 설명, 디버깅, 문서화에 탁월한 성능 발휘



CLAUDE CODE KEY FEATURES

01 지능형 코드 생성 (Code Generation)

자연어 요구사항을 분석하여 시맨틱한 HTML 구조와 모던 CSS 스타일링을 포함한 완성된 코드 블록을 즉시 생성합니다.

02 스마트 리팩토링 (Refactoring)

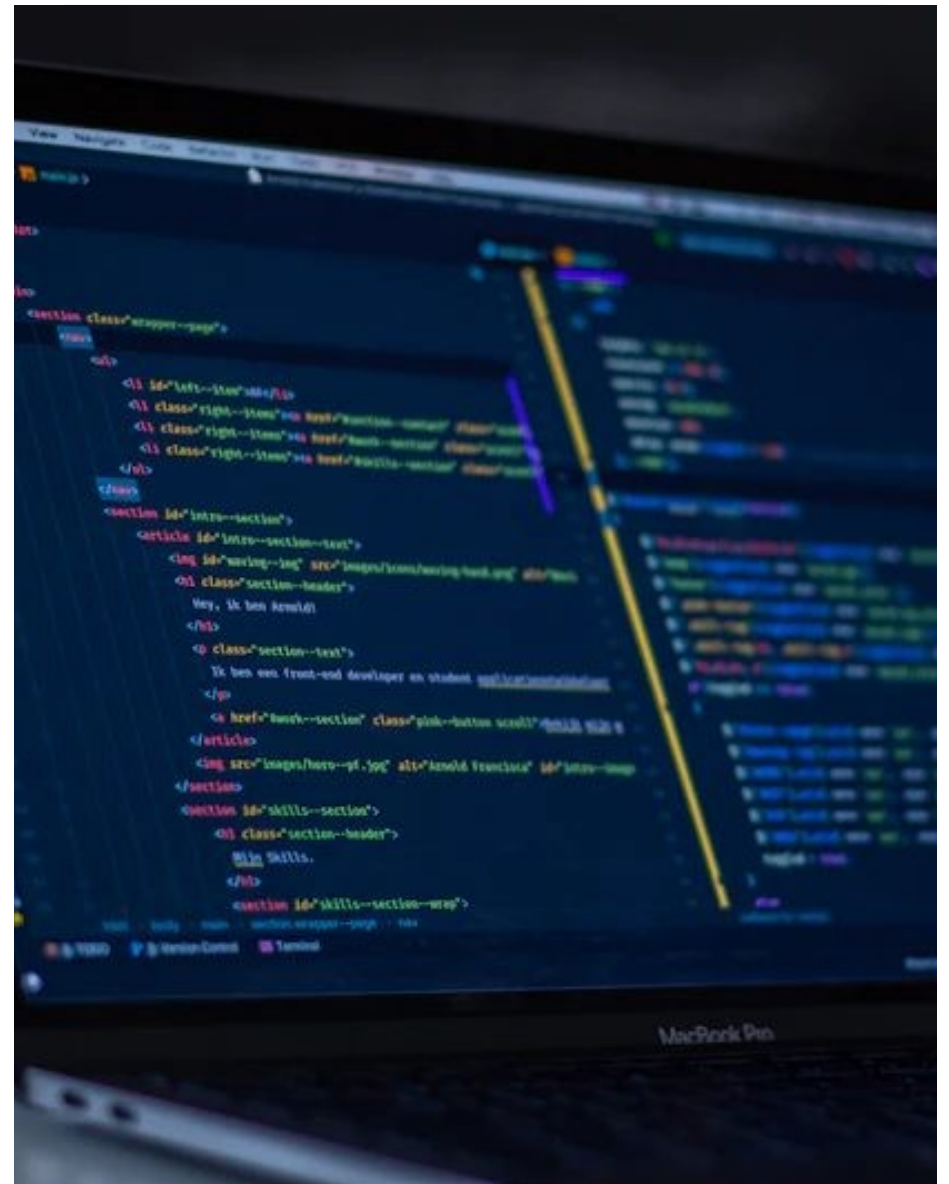
복잡한 로직을 단순화하고, 변수 명명 규칙(Convention)을 통일하며, 코드의 가독성과 유지보수성을 높이는 개선안을 제안합니다.

03 테스트 케이스 자동화 (Test Cases)

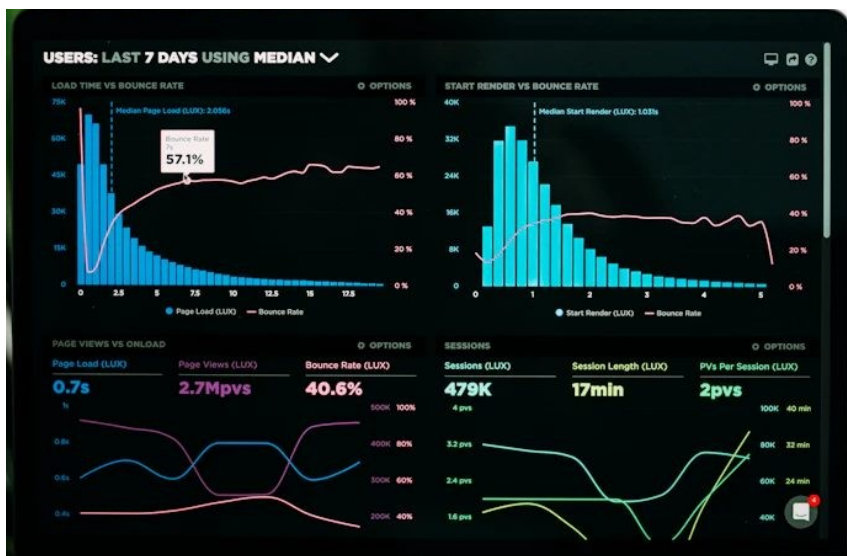
엣지 케이스를 포함한 유닛 테스트 코드를 자동으로 작성하여, 배포 전 잠재적인 버그를 사전에 차단하고 안정성을 확보합니다.

04 코드 설명 및 문서화 (Documentation)

작성된 코드의 작동 원리를 단계별로 설명하고, 프로젝트의 README.md 문서를 자동으로 구성하여 협업 효율을 높입니다.



PROS & CONS



Claude 장점 및 한계 분석

STRENGTHS (장점)



200K+ 토큰의 긴 컨텍스트 윈도우 이해

Context



명확한 설명과 논리적인 코드 리팩토링 능력

Logic



Constitutional AI 기반의 높은 안전성과 신뢰도

Safety

LIMITATIONS (한계)



실시간 웹 검색 기능의 제한 (Code 도구 기준)

No Search



때로는 과도하게 엄격한 안전 필터 작동

Filters

REAL-WORLD EXAMPLES

CLAUDE SCENARIOS



Claude는 단순한 코드 완성을 넘어, 복잡한 로직 설계부터 기존 코드 개선까지 개발 전 과정에서 강력한 파트너가 됩니다.

- 📁 **신규 컴포넌트 설계:** 요구사항을 바탕으로 모듈화된 구조 제안
- ✏️ **레거시 코드 리팩토링:** 가독성 향상 및 최신 문법 적용
- 🔍 **지능형 디버깅:** 에러 로그 분석을 통한 원인 파악 및 해결책 제시
- 📄 **문서 자동화:** 복잡한 로직에 대한 명확한 주석 및 README 작성

Claude Prompt Patterns

효과적인 프롬프트 작성을 위한 3가지 핵심 전략



역할 부여 (Persona)

AI에게 구체적인 전문가 페르소나를 부여하여 답변의 품질, 깊이, 톤앤매너를 설정합니다.

EXAMPLE

"당신은 **10년차 시니어 프론트엔드 개발자**입니다. 클린 코드 원칙과 최신 웹 표준을 준수하여 답변해주세요."



명확한 제약 (Constraints)

사용할 기술 스택, 라이브러리 제한, 코드 스타일 등 기술적 제약 조건을 명확히 나열합니다.

EXAMPLE

"외부 라이브러리 없이 **순수 CSS와 Vanilla JS만 사용**하세요. 모바일 우선(Mobile-first) 반응형으로 작성해주세요."



출력 형식 (Format)

원하는 결과물의 형태를 구체적으로 지시하여 불필요한 서론이나 설명을 줄이고 핵심만 얻습니다.

EXAMPLE

"설명은 생략하고 **완성된 HTML/CSS 코드 블록만 출력**해주세요. 각 주요 섹션에는 주석을 달아주세요."

CLAUDE DEMO

설명형 코드 생성 예시

> PROMPT INPUT



"프론트엔드 멘토로서, 시맨틱 태그를 사용한
간단한 포트폴리오의

'About Me' 섹션 HTML 코드 를 작성해줘.

접근성(ARIA) 속성을 반드시 포함하고,
각 주요 태그에 대한 주석 설명을 덧붙여줘."

💡 PRO TIP

원하는 폴더 구조(예: components/)나
스타일 가이드(예: Tailwind Class)를 함께 제공하면
더 정확한 결과를 얻을 수 있습니다.

</> GENERATED CODE (INDEX.HTML)

```
index.html

<!-- About Me Section -->
<!-- 접근성을 위해 aria-labelledby 속성으로 제목과 섹션을 연결합니다 -->
<section id="about" class="about-section" aria-labelledby="about-title">
  <!-- 컨테이너: 중앙 정렬 및 패딩 -->
  <div class="container">
    <h2 id="about-title" class="section-title">About Me</h2>

    <!-- 프로필 이미지: 장식용 이미지가 아니므로 alt 텍스트 필수 -->
    

    <div class="bio-text">
      <p>노련하세요, 사용자 경험을 중시하는...</p>
    </div>
  </div>
</section>
```

CLAUDE BEST PRACTICES

성공적인 AI 바이브코딩을 위한
5가지 핵심 실천 가이드입니다.



01

📁 입력 컨텍스트 강화

원하는 결과를 얻기 위해 파일 구조, 스타일 가이드, 예시 코드 등 풍부한 맥락 정보를 함께 제공하세요.

02

🔄 작은 단위 반복 (Iteration)

한 번에 완벽한 코드를 기대하기보다, 작은 기능 단위로 생성하고 검증하며 점진적으로 완성하세요.

03

🔪 테스트 우선 (Test First)

AI에게 구현을 요청하기 전에
테스트 케이스나 명확한 검증 기준을
먼저 제시하여 품질을 확보하세요.

04

🛡️ 안전 및 저작권 주의

API 키나 민감한 개인정보를 입력하지 않도록
주의하고, 생성된 코드의 라이선스를 확인하세요.

05

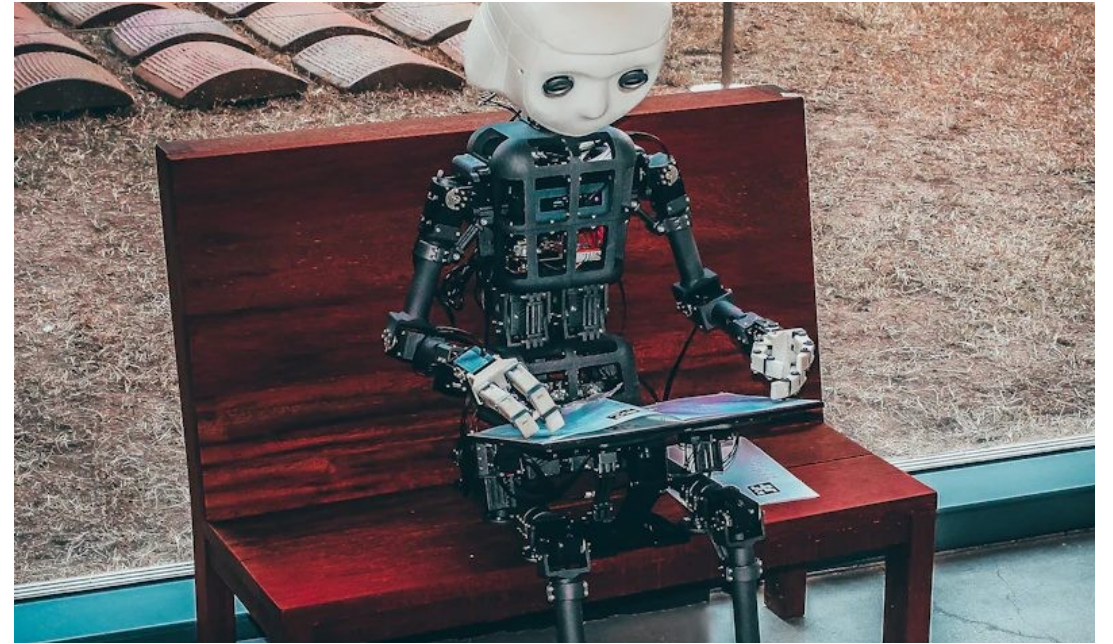
👤 결과물 인간 검수 (Human in the Loop)

AI는 완벽하지 않습니다. 생성된 코드는 반드시 개발자가 직접 리뷰하고,
논리적 오류나 보안 취약점을 검토해야 합니다.

SECTION 03

MEET GEMINI

Google 생태계와 완벽하게 통합된 멀티모달 AI.
코드 제안, 이미지 이해, 최신 정보 검색 연계로
더 넓은 가능성을 경험하세요.



GOOGLE

03

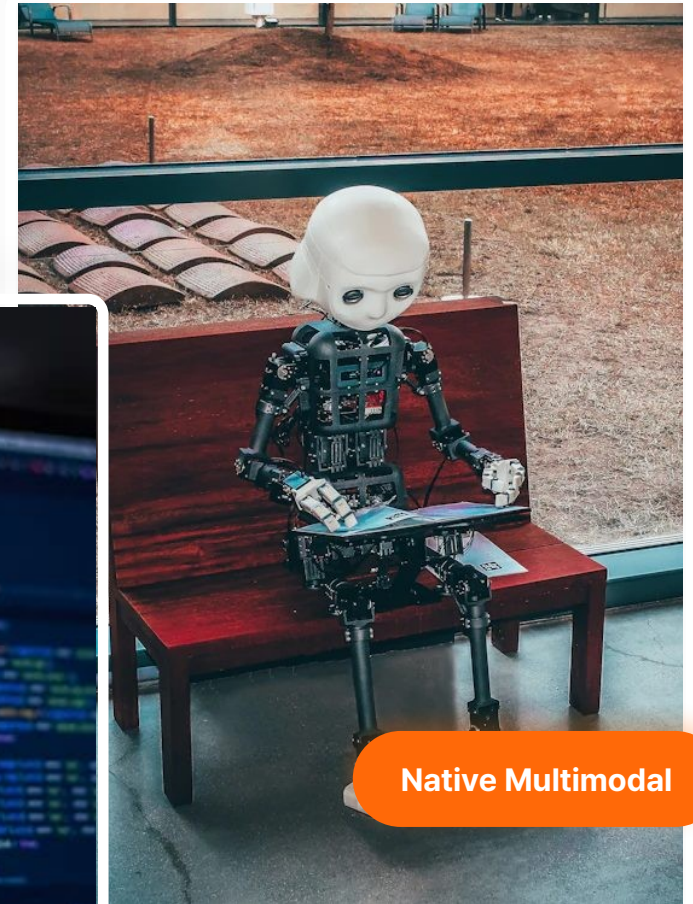
Gemini란? Google의 멀티모달 AI

INTRODUCTION

"텍스트, 이미지, 오디오, 비디오를
동시에 이해하고 추론하는 차세대 모델"

Gemini는 Google이 개발한 최신 AI 모델로,
처음부터 멀티모달로 설계되었습니다. Google Workspace 및 Cloud
생태계와 강력하게 통합되어 있습니다.

- ✓ 멀티모달 이해 및 고도화된 추론 능력
- ✓ Google Workspace · Cloud · Vertex AI 연동
- ✓ 코드, 문서, 이미지가 혼합된 복합 과제 해결



Native Multimodal

GEMINI CODE ASSIST

01 지능형 코드 자동완성

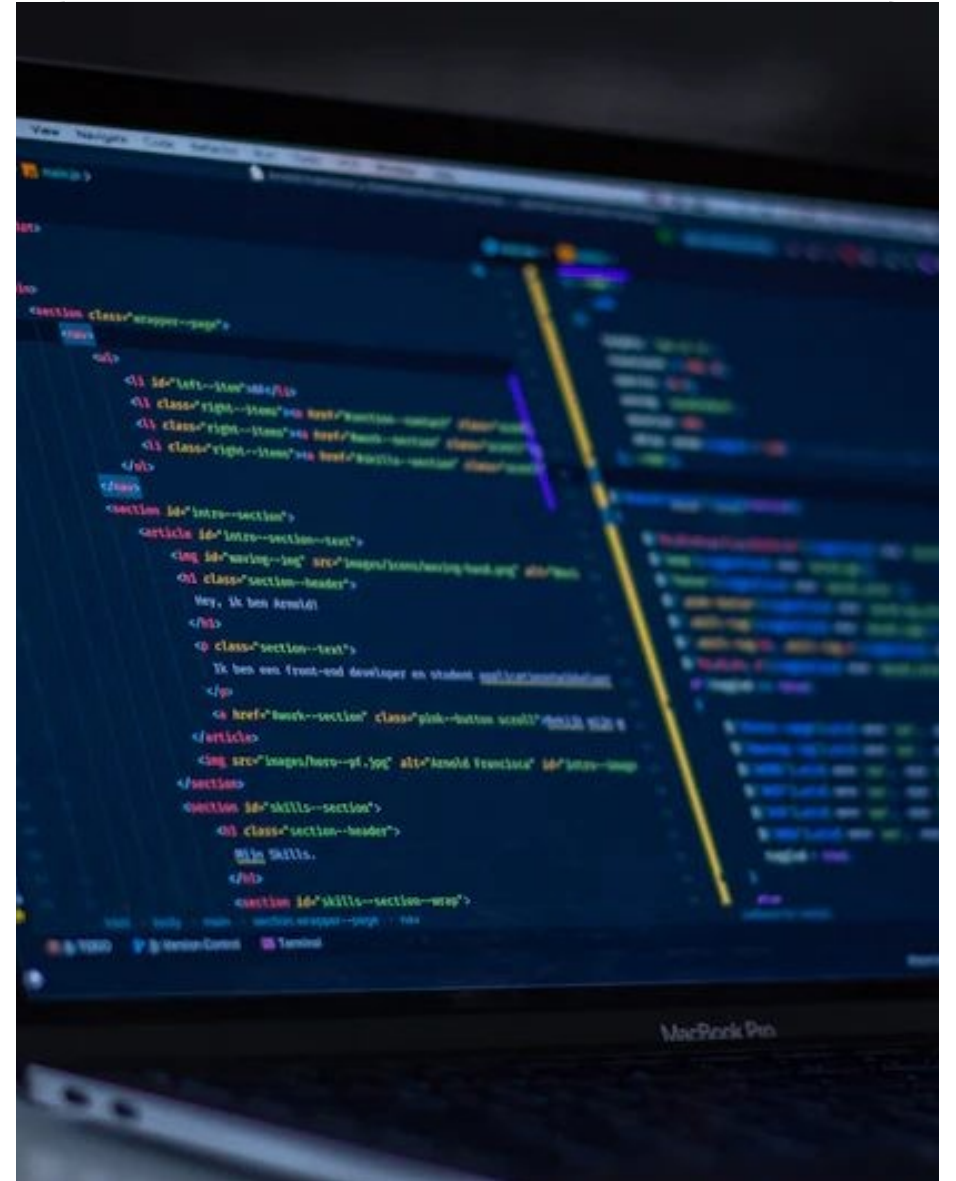
현재 작성 중인 파일의 문맥과 프로젝트 전체 구조를 파악하여 변수, 함수, 그리고 전체 로직을 실시간으로 제안합니다.

02 자연어 명령 코딩

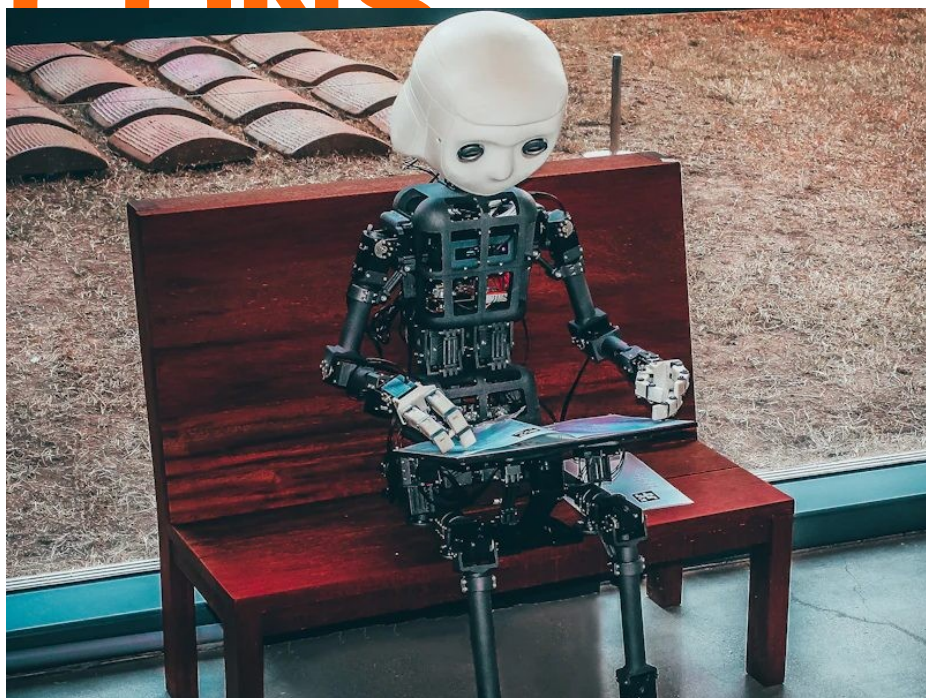
주석으로 원하는 기능을 설명하면(예: "모바일 메뉴 토글 함수"), Gemini가 이를 이해하고 즉시 실행 가능한 코드로 변환해줍니다.

03 IDE 통합 채팅 지원

에디터를 벗어나지 않고 코드에 대한 질문, 리팩토링 요청, 에러 수정 제안을 대화형 인터페이스로 바로 해결할 수 있습니다.



GEMINI PROS & CONS



장점 및 한계 비교 분석



PRO	멀티모달 네이티브 (이미지/영상 인식)	Native Multimodal
PRO	Google 생태계 통합 (Drive/Docs 연동)	Google Ecosystem
PRO	최신 정보 검색 및 반영 속도	Real-time Info
CON	상대적으로 짧은 컨텍스트 (vs Claude)	Context Limit
CON	코드 설명의 깊이와 디테일 부족	Code Depth
CON	복잡한 추론 시 환각(Hallucination) 가능성	Consistency

03

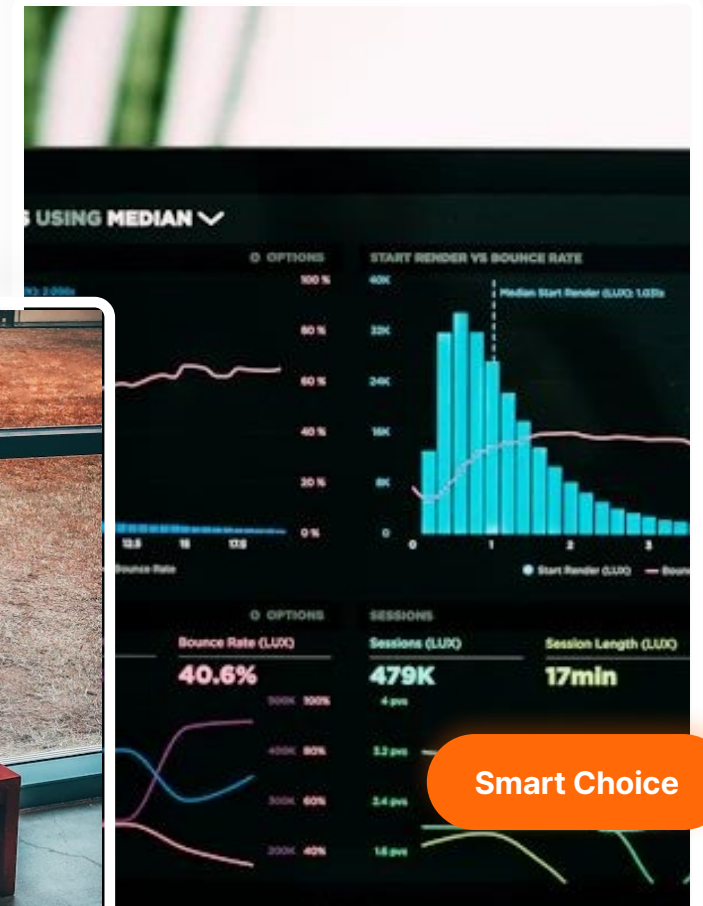
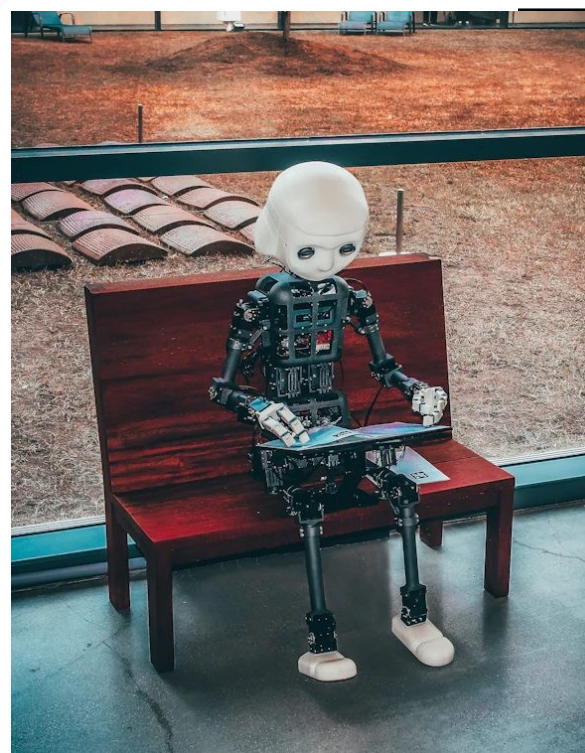
Claude vs Gemini 비교 및 선택 가이드

COMPARISON

"코드 설명과 문맥 파악은 Claude,
멀티모달과 생태계 연동은 Gemini."

두 AI 도구는 각기 다른 강점을 가지고 있습니다.
프로젝트의 성격과 현재 작업 단계에 맞춰 최적의 도구를
선택하는 것이 중요합니다.

-  **Claude:** 장문맥 처리, 상세한 설명, 안전성 우수
-  **Gemini:** 이미지 인식, 구글 연동, 최신 정보 검색
-  **Hybrid:** 복합 과제 시 두 도구의 교차 검증 권장



OPTIMAL COMBINATIONS

TOOL SELECTION



각 AI 도구의 강점을 정확히 이해하고 상황에 맞춰 선택하거나 조합하면 개발 생산성을 극대화할 수 있습니다.

-  **설명 & 리뷰 중심:** 깊이 있는 코드 분석과 논리적 설명이 필요할 땐 **Claude**
-  **자료 & 이미지 연동:** 디자인 시안 분석이나 최신 문서 검색엔 **Gemini**
-  **복합 과제 (Hybrid):** Claude로 구조를 설계하고 Gemini로 구현 디테일 보완

Gemini Prompt Patterns

멀티모달 입력을 활용한 코딩 효율 극대화 전략



이미지 기반 코드 생성

UI 디자인 시안이나 손으로 그린 스케치 이미지를 업로드하여 HTML/CSS 구조 코드를 즉시 생성합니다.

EXAMPLE

"이 **대시보드 디자인 스크린샷**을 분석해서 반응형 HTML 구조와 Tailwind CSS 클래스로 변환해주세요."



문서 참조 코딩

PDF 문서나 최신 API 가이드 문서를 첨부하여, 학습되지 않은 최신 정보 기반의 정확한 코드를 요청합니다.

EXAMPLE

"첨부된 **API 명세서 파일**을 참조하여 사용자 로그인 컨트롤러(Controller) 코드를 작성하세요."



시각적 디버깅

에러 로그 캡처 화면이나 UI가 깨진 화면을 보여주며 문제의 원인을 시각적으로 분석하고 해결책을 찾습니다.

EXAMPLE

"이 **에러 로그 캡처 화면**을 보고 어떤 모듈 충돌이 발생했는지 분석하고 해결 패치를 제안해주세요."

GEMINI DEMO

이미지 기반 코드 생성 예시

MULTIMODAL PROMPT



design_mockup_v1.png

1.2 MB

"업로드한 디자인 스크린샷을 바탕으로 메인 컬러와 폰트 스타일을 추출해서 **CSS 변수 (Root)**로 정리해줘.

또한, 모바일 우선(Mobile-first) 접근 방식으로 미디어 쿼리 기본 골격도 함께 작성해줘."

GEMINI TIP

이미지 내의 텍스트가 명확하거나 레이아웃 구조가 뚜렷할수록 Gemini가 더 정확한 CSS 구조를 제안합니다. 복잡한 UI는 부분을 잘라서 질문하세요.

</> GENERATED CODE (STYLES.CSS)



styles.css

```
:root {  
  /* 이미지에서 추출한 색상 팔레트 */  
  --primary-color: #FF6908;  
  --text-main: #111827;  
  --bg-light: #F9FAFB;  
  
  /* 타이포그래피 설정 */  
  --font-base: 'Pretendard', sans-serif;  
  --spacing-md: 16px;  
}  
  
/* Mobile First 기본 스타일 */  
.container {  
  width: 100%;  
  padding: var(--spacing-md);  
}  
  
/* 태블릿 및 데스크톱 확장 (768px 이상) */  
@media (min-width: 768px) {  
  .container {  
    max-width: 720px;  
    margin: 0 auto;  
  }  
}
```

AI TOOL PRACTICE

평가 기준 (Evaluation Criteria)

- ✓ **정확성:** 요청한 기능이 오류 없이 동작하는가?
- ✓ **가독성:** 코드가 깔끔하고 시맨틱 태그를 사용하는가?
- ✓ **접근성:** 웹 접근성 표준(WAI-ARIA 등)을 준수하는가?



TASK

Claude vs Gemini 실습 과제

동일한 프롬프트로 HTML 골격과
CSS 카드 레이아웃을 생성하여 결과를 비교합니다.

HTML 골격 생성 및 검증

> PROMPT INPUT



"웹 표준을 준수하는 포트폴리오 사이트의
시맨틱 HTML 골격을 작성해줘.

필수 포함 요소:

1. **header, nav, main, footer** 시맨틱 태그
2. 웹 접근성을 위한 **WAI-ARIA 랜드마크** 역할
3. 모바일 대응을 위한 meta viewport 설정"

✓ 검증 포인트 (Validation)

Live Server 확인:

브라우저에서 구조가 깨지지 않고 렌더링되는지 확인

WAI-ARIA: `role="banner"`, `role="main"`

등 랜드마크 역할 적용 여부

시맨틱 구조: `<div>` 남용 대신 의미 있는 태그 사용 여부

</> GENERATED CODE (INDEX.HTML)

```
index.html
<!DOCTYPE html> <html lang="ko"> <head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Portfolio</title>
</head>
<body>
  <!-- 접근성: 본문 바로가기 링크 -->
  <a href="#main-content" class="skip-link">본문으로 건너뛰기 </a>

  <!-- 헤더 영역 -->
  <header role="banner">
    <div class="logo">  </div>
    <nav role="navigation" aria-label="메인 메뉴">
      <ul>
        <li><a href="#about">소개 </a></li>
        <li><a href="#projects">프로젝트 </a></li>
        <li><a href="#contact">연락처 </a></li>
      </ul>
    </nav>
  </header>

  <!-- 메인 콘텐츠 영역 -->
  <main id="main-content" role="main">
    <!-- 섹션들이 여기에 들어갑니다 -->
  </main>

  <footer role="contentinfo">
    <p>© 2026 Developer Kim. All rights reserved. </p>
  </footer>
</body> </html>
```

✏ 코드 개선 (Refinement)

접근성을 높이기 위해 로고에 대체 텍스트(alt)를 구체적으로 작성하고, 키보드 사용자를 위한
'본문 바로가기(Skip Link)'를 `<nav>` 내부에 추가하도록 요청하세요.

CSS 카드 레이아웃 실습

> PROMPT INPUT



"반응형 웹 디자인 전문가로서, 포트폴리오 프로젝트 섹션에 사용할

CSS Grid 카드 레이아웃 코드를 작성해줘.

1. 데스크탑 3열, 모바일 1열 자동 조정 (auto-fit 사용)
2. 카드 Hover 시 부드러운 떠오름 효과 (transform)와 그림자 추가
3. 접근성을 위한 포커스 스타일 포함"

💡 PRO TIP

Gemini는 CSS Grid의 최신 스펙인 repeat(auto-fit, minmax(...)) 패턴을 잘 이해합니다. 미디어 쿼리 없이도 반응형을 구현하는 것이 핵심입니다.

</> GENERATED CODE (STYLE.CSS)

style.css

```
/* Projects Grid Layout */
.projects-grid {
  display : grid ;
  /* 300px 이상 공간이 있으면 자동으로 열을 채움 (반응형) */
  grid-template-columns : repeat( auto-fit , minmax(300px, 1fr ));
  gap: 2rem;
  padding: 2rem 0;
}

/* Project Card Style */
.project-card {
  background var(--bg-card) ;
  border-radius : 12px;
  box-shadow 0 4px 6px rgba(0,0,0,0.05) ;
  transition : transform 0.3s ease, box-shadow 0.3s ease;
}

/* Hover & Focus Effects */
.project-card:hover ,
.project-card:focus-visible {
  transform : translateY (-8px);
  box-shadow 0 12px 20px rgba(0,0,0,0.1) ;
  outline : 2px solid #FF6900; /* 접근성 포커스 링 */
}
```

PRACTICE EXAMPLE 01

Sticky Navbar 구현 – part 1

> PROMPT INPUT



"스크롤 시 상단에 고정되는 **Sticky Navbar**를 구현해줘. CSS는 `position: sticky`를 사용하고,

자바스크립트로 **IntersectionObserver**를 활용하여 현재 보고 있는 섹션에 해당하는 메뉴가 자동으로 하이라이트(active 클래스 추가)

되도록 해줘."

💡 PRO TIP

기존의 scroll 이벤트 리스너 대신 **IntersectionObserver**를 사용하면 메인 스템드의 부하를 줄여 렌더링 성능을 크게 최적화할 수 있습니다.

</> GENERATED CODE (SCRIPT.JS)

```
// 1. 성능 및 접근성을 고려한 관찰자 설정
const observerOptions = {
  root: null,
  rootMargin: '0px',
  threshold: 0.5
};

// 2. IntersectionObserver 콜백 함수 정의
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      // 현재 화면에 보이는 섹션의 ID 가져오기
      const id = entry.target.getAttribute('id');

      // 네비게이션 링크 활성화 상태 업데이트
      document.querySelector('.nav-link').forEach(link => {
        link.classList.remove('active');
        if (link.getAttribute('href') === `#${id}`) {
          link.classList.add('active');
        }
      });
    }
  });
});
```

PRACTICE EXAMPLE 01

Sticky Navbar 구현 – part 2

> PROMPT INPUT



"스크롤 시 상단에 고정되는 **Sticky Navbar**를 구현해줘. CSS는 **position: sticky**를 사용하고, 자바스크립트로 **IntersectionObserver**를 활용하여 현재 보고 있는 섹션에 해당하는 메뉴가 자동으로 하이라이트(active 클래스 추가) 되도록 해줘."

💡 PRO TIP

기존의 scroll 이벤트 리스너 대신 **IntersectionObserver**를 사용하면 메인 스톱드의 부하를 줄여 렌더링 성능을 크게 최적화할 수 있습니다.

</> GENERATED CODE (SCRIPT.JS)

```
script.js

});
}, observerOptions);

// 3. 모든 섹션 관찰 시작
document.querySelectorAll('section').forEach(section => {
  observer.observe(section);
});

// 4. 접근성 고려: 사용자가 동작 줄이기를 선호하는지 확인
const prefersReducedMotion = window.matchMedia('(prefers-reduced-motion: reduce)');
if (prefersReducedMotion.matches) {
  document.documentElement.style.scrollBehavior = 'auto'; // 스무스 스크롤 비활성화
}
```

Hero 그라데이션 배경

단색 배경을 conic-gradient와 혼합 모드로 개선하여 시각적 깊이감 추가

BEFORE

STATIC

Original Design

Plain Background

```
.hero-section {  
  /* 단조로운 단색 배경 */  
  background-color : #111827;  
  position : relative ;  
  overflow : hidden;  
  color : #ffffff ;  
}  
  
.hero-content {  
  z-index : 10;  
  position : relative ;  
}
```



AFTER

IMPROVED

Enhanced Visual

Gradient Mesh

```
.hero-section {  
  background-color : #111827;  
  /* Conic Gradient를 빛 혼합 추가 */  
  background-image : conic-gradient (  
    from 0deg at 50% 50%,  
    #FF6908 0deg,  
    transparent 60deg  
  );  
  filter : blur ( 100px);  
  opacity : 0.6 ;  
}
```



PRO TIP: 성능 최적화

그라데이션과 blur 효과는 렌더링 비용이 높으므로, will-change: transform을 사용하거나 가상 요소(::before)에 적용하여 본문 리플로우를 방지하는 것이 좋습니다.

실전 예제 3 : 접근성 버튼 상태

> PROMPT INPUT

"모든 사용자가 접근 가능한 버튼 컴포넌트의 CSS 스타일을 작성해줘.

마우스 오버(Hover), 키보드 포커스(:focus-visible), 비활성화(:disabled), 그리고 토글 상태(aria-pressed)에 대한 스타일을 명확히 구분해야 해.

고대비 모드와 스크린 리더 사용자도 고려해 줘."

💡 PRO TIP

:focus 대신 **:focus-visible**을 사용하면 마우스 클릭 시 발생하는 불필요한 아웃라인은 숨기고, 키보드 탐색 시에만 명확한 포커스링을 제공하여 UX를 향상시킬 수 있습니다.

</> GENERATED CODE (STYLE.CSS)

```
style.css

/* 1. Base Button Style */ .btn {
  background-color : #FF6908;
  color : white ;
  border : 2px solid transparent ;
  cursor : pointer ;
  transition : all 0.2s ease ;
}
/* 2. Interaction States (Mouse) */
.btn :hover { background-color : #E65100; }
.btn :focus-visible { transform : translateY(1px) ; }
/* 3. Accessibility: Keyboard Focus Only */ .btn
{
  outline : none;
  box-shadow: 0 0 0 3px rgba(255, 105, 8, 0.4) ;
  border-color : white ; /* 고대비 모드 지원 */
}
/* 4. Toggle State (Aria Attribute) */ .btn[aria-pressed="true"] {
  background-color : #1F2937;
  box-shadow: inset 0 2px 4px rgba(0,0,0,0.2) ;
}
/* 5. Disabled State */ .btn :disabled {
  opacity : 0.6 ;
  cursor : not-allowed ;
}
```

[P4] 실전 예제4 : 모달 컴포넌트 (Focus Trap) - Part 1

> PROMPT INPUT



"웹 접근성 지침(WCAG)을 준수하는 **모달(Modal) 창 컴포넌트**를 구현해줘.

모달이 열릴 때 배경 스크롤은 잠겨야 하고, **ESC 키**를 누르거나 배경을 클릭하면 닫혀야 해. 특히 키보드 포커스가 모달 밖으로 나가지 않도록 **'Focus Trap'** 기능을 반드시 포함해줘."

i NEXT STEP

이 코드는 **Part 1**입니다.

요소 선택과 Tab 키(순방향) 처리를 다루며, **Part 2**에서 Shift+Tab(역방향) 이동과 ESC 키 처리를 이어서 다룹니다.

</> GENERATED CODE (MODAL.JS) - PART 1: 요소 선택 & TAB 처리

modal.js

```
// 1. 포커스 트랩을 위한 요소 선택
const modal = document.getElementById('modal');
const focusableElements = modal.querySelectorAll(
  ':focusable:not([tabindex="-1"])'
);

const firstElement = focusableElements[0];
const lastElement = focusableElements[focusableElements.length - 1];

// 2. 키보드 이벤트 리스너 시작
modal.addEventListener('keydown', function (e) {
  // Tab 키 누를 감지
  if (e.key === 'Tab') {
    // Shift + Tab: 역방향 이동 (첫 요소 -> 마지막 요소)
    if (e.shiftKey) {
      if (document.activeElement === firstElement) {
        e.preventDefault();
        lastElement.focus();
      }
    }

    // Tab: 순방향 이동 (마지막 요소 -> 첫 요소)
    else {
      if (document.activeElement === lastElement) {
        e.preventDefault();
        firstElement.focus();
      }
    }
  }
  // ... (Part 2에서 계속: ESC 키 처리 및 리스너 종료) ...
})
```

[P4] 모달 Focus Trap - Part 2 : Shift+Tab & ESC

CONTEXT



"Part 1에서 Tab 순방향 처리를 다뤘고, Part 2에서는 **Shift+Tab 역방향 처리**와 **ESC 키로 모달 닫기** 로직을 완성합니다.

이를 통해 키보드 사용자가 모달 갇힘(Trap) 없이 자유롭게 탐색할 수 있도록 합니다."

✓ A11Y CHECKPOINT

ESC 키로 모달을 닫는 기능은 WCAG 2.1 Success Criterion 1.4.13 및 키보드 접근성의 필수 요소입니다. 사용자가 언제든지 현재 작업을 취소할 수 있어야 합니다.

</> GENERATED CODE (MODAL.JS - PART 2)

```
modal.js

// ... (이전 코드: if (e.key === 'Tab'))
// 1. Shift + Tab: 뒤로 가기 (첫 요소 -> 마지막 요소)
if ( e.shiftKey ) {
  if ( document.activeElement === firstElement ) {
    e.preventDefault ();
    lastElement .focus();
  }
}

// 2. Tab: 앞으로 가기 (마지막 요소 -> 첫 요소)
else {
  if ( document.activeElement === lastElement ) {
    e.preventDefault ();
    firstElement .focus();
  }
}
} // Tab 키 처리 종료

// 3. ESC 키: 모달 닫기
if ( e.key === 'Escape' ) {
  closeModal(); // 모달 닫기 함수 호출
}
}); // 이벤트 리스너 종료
```

STEP 2: HTML STRUCTURE

index.html 골격 - Part 2 : Footer & Closing

PREVIOUS CONTEXT



"Part 1에서 **HEAD + HEADER + MAIN** 구조를 다뤘고, Part 2에서는 **Footer**와 문서 닫기 태그를 완성합니다."

주요 구성:

- Main 영역 종료
- 저작권 및 소셜 링크가 포함된 Footer
- JavaScript 파일 연결
- HTML 문서 종료

CHECK POINT

<footer> 태그 안에 저작권 정보(©)와 소셜 링크를 포함하여 페이지의 마무리를 명확히 합니다. 스크립트 태그는 body 닫기 직전에 배치하여 로딩 성능을 최적화하세요.

</> GENERATED SKELETON (INDEX.HTML) - PART 2

```
index.html

<!-- ...Part 1 (Header, Main Content)에서 이어질... -->

<!-- Main: 주요 콘텐츠 영역 종료 -->
</main>

<!-- Footer: 저작권 및 연락처 정보 -->
<footer class="site-footer" role="contentinfo">
  <div class="container">
    <p>&copy; 2026 My AI Portfolio. All rights reserved. </p>
    <div class="social-links">
      <a href="#" aria-label="GitHub Profile">GitHub</a>
      <a href="#" aria-label="LinkedIn Profile">LinkedIn</a>
    </div>
  </div>
</footer>

<!-- Scripts: 페이지 로딩 성능을 위해 하단 배치 -->
<script src="/js/main.js"></script>

</body>
</html>
```

다크모드 토글 - Part 2 : Toggle Event

LOGIC FLOW



Part 1에서 요소 선택과 초기 테마 설정을 다뤘고, Part 2에서는 토글 버튼 클릭 이벤트와 **localStorage** 저장 로직을 완성합니다.

사용자가 버튼을 클릭할 때마다 data-theme 속성을 변경하고, 이를 로컬 스토리지에 즉시 동기화하여 설정을 유지합니다.

PERSISTENCE TIP

localStorage.setItem()은 페이지를 새로고침하거나 브라우저를 닫았다 열어도 설정이 유지되도록 합니다. UX 측면에서 개인화 설정 저장에 필수적입니다.

</> GENERATED CODE (SCRIPT.JS)

```
// ... (Part 1: 요소 선택 및 초기 테마 설정 코드) ...

// 3. 토글 이벤트 리스너 추가
toggleBtn. addEventListener ( 'click' , () => {
  // 3-1. 현재 테마 값 확인 (html 태그의 data-theme 속성)
  const currentTheme = root. getAttribute ( 'data-theme' );

  // 3-2. 새 테마 결정 (dark이면 light로, light이면 dark로 전환)
  const newTheme = currentTheme === 'dark' ? 'light' : 'dark' ;

  // 3-3. DOM에 새 테마 적용
  root. setAttribute ( 'data-theme' , newTheme);

  // 3-4. 로컬 스토리지에 저장 (영구 보존)
  localStorage. setItem ( 'theme' , newTheme);

  // (옵션) 아이콘 변경 등의 UI 업데이트 로직을 여기에 추가할 수 있습니다.
  console. log ( `Theme switched to: ${newTheme}` );
});

// End of script.js
```

고급 기능 추가

프로젝트 필터링 - Part 2 : Filter Logic

> PROMPT INPUT



"Part 1에서 버튼 이벤트와 활성화 상태 업데이트를 다뤘고, Part 2에서는 실제 프로젝트 카드 표시/숨김 필터링 로직을 완성합니다."

💡 PRO TIP

DOM 조작을 최소화하기 위해 CSS 클래스(.hide)를 토글하는 방식을 사용하면 성능이 더 좋고, 페이드 애니메이션을 추가하기도 쉽습니다.

</> GENERATED CODE (SCRIPT.JS)

```
script.js

// ...Part 1 코드(버튼 이벤트 시작)에 이어서...

// 3. 프로젝트 필터링 로직
projects.forEach(card => {
  // 카드의 data-category 속성 값과 선택된 필터 값 비교
  if (filterValue === 'all' || card.dataset.category === filterValue) {
    // 조건에 맞으면 카드 표시
    card.style.display = 'block' ;
    // 애니메이션을 위한 클래스 추가 (선택사항)
    card.classList.add('fade-in' );
  } else {
    // 조건에 맞지 않으면 카드 숨김
    card.style.display = 'none' ;
    card.classList.remove('fade-in' );
  }
}); // projects.forEach 종료
}); // btn.addEventListener 종료
}); // filterBtns.forEach 종료
```

PRACTICAL EXAMPLE 05

API Fetch 모킹 & 상태 관리 - Part 1: Try 블록

> PROMPT INPUT



"JSONPlaceholder API(/users)를 사용하여 사용자 데이터를 가져오는 JavaScript 비동기 함수를 작성해줘."

데이터를 가져오는 동안의 로딩>Loading) 상태와 실패 시의 에러>Error) 처리 로직을 반드시 포함해줘."

💡 PART 1 GUIDE

이 슬라이드는 **Part 1**으로, 상태 초기화부터 성공적인 데이터 페칭(Try 블록)까지를 다룹니다. 에러 처리(Catch)와 상태 정리(Finally)는 다음 슬라이드(Part 2)에서 이어집니다.

</> GENERATED CODE (API.JS) - PART 1

```
api.js

// 상태 변수 초기화
let isLoading = false;
let error = null;
let users = [];

// 사용자 데이터 가져오기 함수
async function fetchUsers() {
  isLoading = true;
  error = null;

  try {
    // JSONPlaceholder Mock API 호출
    const response = await fetch('https://jsonplaceholder.typicode.com/users');

    // 응답 상태 확인 (200-299 범위가 아니면 에러)
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    // 데이터 파싱 및 클라이언트
    users = await response.json();
    console.log('Data fetched successfully:', users);
  } catch & finally 블록은 Part 2에서 이어집니다... */
}
```

PRACTICAL EXAMPLE 05

API Fetch 모킹 & 상태 관리 - Part 2: Error Handling

> PROMPT INPUT



"이전 Part 1의 try 블록에 이어서, 에러가 발생했을 때(catch) 사용자에게 메시지를 보여주는 처리 로직과, 요청 성공/실패 여부와 상관없이 로딩 상태를 종료하는(finally) 코드를 작성해줘."

💡 PRO TIP

실제 프로덕션 환경에서는 사용자 친화적인 에러 메시지를 표시하고, 필요한 경우 에러 로깅 서비스(Sentry 등)로 전송하는 것이 좋습니다.

</> GENERATED CODE (API.JS) - PART 2

```
api.js

// ... (Part 1: try 블록 이후)

catch (err) {
  // 4. 에러 처리 로직
  console.error('Failed to fetch users:', err);
  error = err.message

  // 사용자에게 에러 메시지 표시 (DOM 조작 예시)
  const userList = document.getElementById('user-list');
  if (userList) {
    userList.innerHTML += `<div class="error-msg">Error: ${error}. Please try again later.</div>`;
  }
} finally {
  // 5. 로딩 상태 종료 (성공/실패 무관하게 실행)
  isLoading = false;
  console.log('Fetch operation completed. Loading state:', isLoading);

  // 로딩 스피너 숨기기
  const spinner = document.querySelector('.loading-spinner');
  if (spinner) {
    spinner.style.display = 'none';
  }
}
// End of fetchUsers function
```

CSS GRID LAYOUT

상세 구현 1 : About 타임라인

> PROMPT INPUT



"About Me 섹션에 내 경력을 보여줄
수직 타임라인을 CSS Grid로 만들어줘.

구조는 [연도] - [구분선] - [상세내용]

형태의 3열 그리드여야 하고, 모바일에서는 1
열로 변경되어야 해."

💡 PRO TIP

grid-template-columns: 1fr auto 4fr; 패턴을
사용하면 타임라인의 중심축(auto)을 기준으로 양쪽
콘텐츠 비율을 쉽게 제어할 수 있습니다.

</> GENERATED CODE (STYLE.CSS)

```
style.css

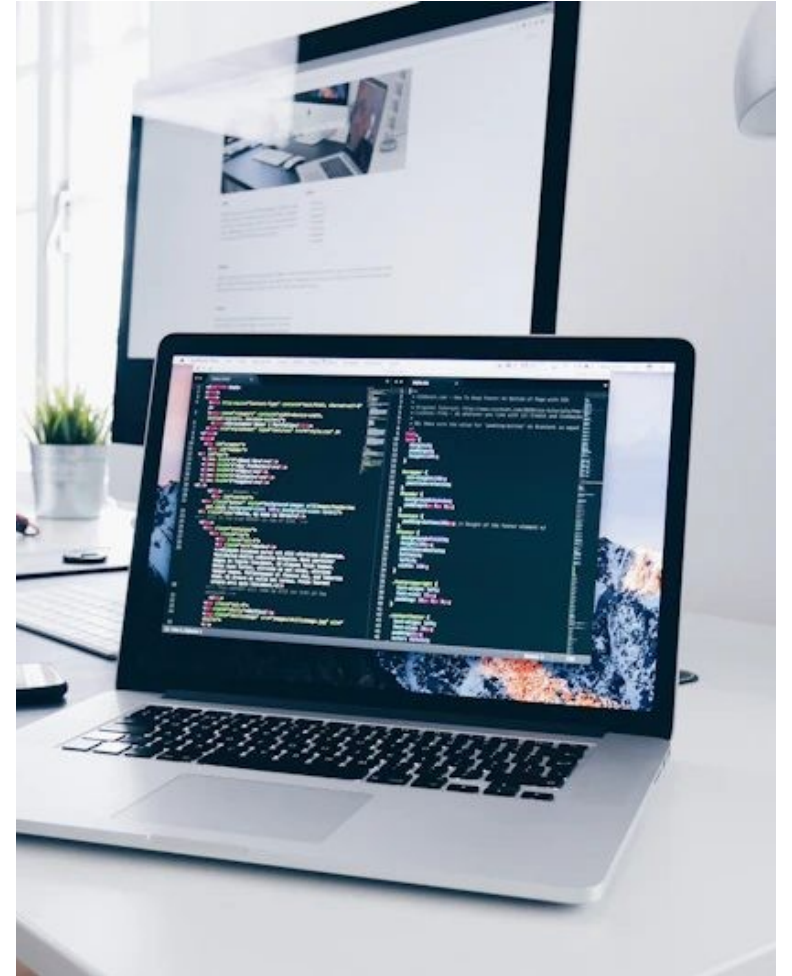
/* 타임라인 그리드 레이아웃 */
.timeline-container {
  display : grid ;
  /* 연도(1fr) - 구분선(자동) - 내용(3fr) */
  grid-template-columns : 1fr auto 3fr ;
  gap: 2rem;
  align-items : start ;
} /* 구분선 (Timeline Line) */
.timeline-divider {
  width : 2px;
  background-color : var(--primary-color) ;
  height : 100%;
  position : relative ;
} /* 반응형: 모바일에서는 1열로 변경 */
@media(max-width : 768px) {
  .timeline-container {
    grid-template-columns : 1fr ;
    padding-left : 1.5rem;
    border-left : 2px solid var(--primary-color) ;
  }
  .timeline-divider { display : none; }
}
```

클로드코드와 제미나이를 이용한 바이트코딩 포트폴리오 배포

AI와 함께하는 초고속 웹 개발 마스터 클래스
환경 설정부터 GitHub Pages 실전 배포까지

강사

이 애 본 (aebon@hs.ac.kr)
한신대학교 AI.SW대학



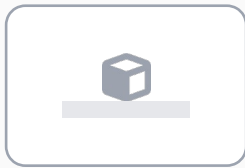
카드 Hover 모션 최적화

Reflow를 유발하는 속성 대신 Composite 레이어 속성 활용으로 60fps 달성

BEFORE

LOW FPS

Layout Thrasing



```
.project-card {
  transition :all 0.2s ;
}

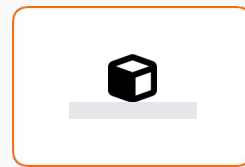
.project-card:hover {
  /* Bad: 주변 요소 레이아웃에 영향 */
  margin-top : -10px;
  width : 102%;
  /* Bad: 비용이 높은 속성 */
  box-shadow 0 5px 15px rgba(0,0,0,0.2) ;
}
```

→

AFTER

GPU ACCELERATED

Composite Only



```
.project-card {
  /* 성능 힌트 제공 */
  will-change :transform ;
  transition :transform 0.3s ease-out ;
}

.project-card:hover {
  /* GPU 가속 활용 (레이아웃 영향 없음) */
  transform : translateY (-8px);
  box-shadow 0 12px 24px rgba(0,0,0,0.15) ;
}
```



PRO TIP: will-change 사용 시 주의사항

will-change: all과 같이 남용하면 메모리 사용량이 급증할 수 있습니다. 애니메이션이 적용되는 요소에만 제한적으로 사용하고, 애니메이션이 끝나면 제거하는 것이 이상적입니다.

PROJECT IMPLEMENTATION

상세 구현3 : 태그 필터 칩

> PROMPT INPUT

"프로젝트 섹션에 **다중 선택이 가능한 태그 필터링** 기능을 JavaScript로 구현해줘.

1. HTML: 각 필터 버튼에 접근성을 위해 `role="checkbox"`와 `aria-checked` 속성을 사용해줘.

2. JS: `data-tags` 속성을 기반으로 선택된 태그가 모두 포함된(AND 조건) 프로젝트만 보이도록 로직을 작성해줘."

💡 LOGIC TIP

단일 필터링이 아닌 '다중 조건' 필터링 시에는 Array 메서드인 `every()` (AND 조건) 또는 `some()` (OR 조건)을 활용하면 코드가 훨씬 간결해집니다.

</> GENERATED CODE (MAIN.JS)

main.js & html structure

```
<!-- Filter UI Structure --> <button class ="filter-chip" role ="checkbox" aria-checked ="false" data-filter ="react" >React</button>
<!-- Project Card Example --> <div class ="project-card" data-tags ="react css" >... </div>

// Filter Logic
const chips =document.querySelectorAll ( '.filter-chip' );
const cards =document.querySelectorAll ( '.project-card' );

chips .forEach ( chip =>{
  chip .addEventListener ( 'click' , () =>{
    // 1. Toggle State & Accessibility
    const isChecked=chip .getAttribute ( 'aria-checked' ) ==='true' ;
    chip .setAttribute ( 'aria-checked' , ! isChecked);
    chip .classList .toggle ( 'active' );
    updateGallery ();
  });
});

function updateGallery () {
  // 2. Get Active Filters
  const activeFilters = Array .from( document.querySelectorAll ( '.filter-chip[aria-checked="true"]' ))
    .map( c => c.dataset.filter);

  cards .forEach ( card =>{
    const tags = card .dataset.tags. split ( ' ' );
    // AND Condition: Must have ALL active filters
    const isVisible = activeFilters .every ( f => tags .includes ( f ));
    card .style.display = isVisible ? 'block' : 'none' ;
  });
}
```

Form Submission Logic

Formspree 설정, curl 테스트, 성공/실패 토스트 UI 구현

1



Formspree 설정

계정 생성 및 New Project
엔드포인트(URL) 획득

```
→ Login formspree.io
→ Create New Form
https://formspree.io/f/x...
```

2



HTML Form 연결

Method="POST" 설정 및
Honeypot(스팸방지) 추가

```
<form action="{URL}"
method="POST">
<!-- Spam Protection -->
<input name="_gotcha"...
```

3



CURL 테스트

터미널에서 직접 요청하여
수신 동작 검증

```
$ curl -X POST
-d "email=test@a.com"
https://formspree.io/f/x..
```

4



피드백 UI 구현

Fetch API 응답에 따라
성공/실패 토스트 표시

```
const res = await fetch()
if (res.ok) {
  showToast("Success!")
}
```

LIGHTHOUSE

90+ SCORE

STRATEGY

웹사이트 품질의 4가지 핵심 지표를
최적화하여 사용자 경험과
검색 엔진 순위를 극대화하세요.

01 🚀 성능 (Performance)

이미지 포맷(AVIF/WebP) 최적화, 불필요한 JS 제거, 브라우저 캐싱 정책을 적용하여 로딩 속도를 개선하세요.

02 ♿ 접근성 (Accessibility)

시맨틱 HTML 구조 사용, 이미지 대체 텍스트 제공, 적절한 명도 대비를 준수하여 모든 사용자를 포용하세요.

03 ✔️ 베스트 프랙티스

HTTPS 사용, 보안 취약점 라이브러리 제거, 올바른 이미지 비율 설정을 통해 웹 표준 안전성을 확보하세요.

04 🔍 검색 엔진 최적화 (SEO)

메타 태그(Title, Description) 최적화, sitemap.xml 제출, robots.txt 설정으로 검색 가시성을 높이세요.

05 📈 Core Web Vitals 집중 관리

LCP(최대 콘텐츠 렌더링), FID(최초 입력 지연), CLS(누적 레이아웃 이동) 3대 지표를 중점적으로 모니터링하고 개선하여 사용자 경험을 완성하세요.

반응형 점검 체크포인트

다양한 디바이스 환경에서 완벽한 사용자 경험을 제공하기 위한 4단계 검증

**320px****MOBILE S**

아이폰 SE 등 소형 모바일 기기 기준의 최소 너비 검증.

- ✓ 가로 스크롤 발생 여부 확인
- ✓ 긴 단어/URL 줄바꿈(word-break) 처리
- ✓ 패딩 최소화로 콘텐츠 공간 확보

**480px****MOBILE**

대부분의 최신 스마트폰 세로 모드 기준.

- ✓ 햄버거 메뉴 및 모바일 네비게이션 작동
- ✓ 터치 타겟 크기(최소 44x44px) 확보
- ✓ 폰트 가독성(본문 16px 이상) 확인

**768px****TABLET**

아이패드 등 태블릿 세로 모드 및 대형 폰 가로 모드.

- ✓ 그리드 레이아웃 전환(1열 → 2열)
- ✓ 이미지 비율 및 해상도 최적화
- ✓ 여백(Spacing) 적절성

**1024px+****DESKTOP**

노트북 및 데스크탑 모니터 환경.

- ✓ 네비게이션 메뉴 확장(햄버거 → 링크)
- ✓ Hover 효과 및 마우스 인터랙션
- ✓ 최대 너비(max-width) 제한 및 중앙 정렬

i DPR(Device Pixel Ratio) 고려 : 레티나 디스플레이 등 고해상도 화면에서는 이미지가 흐릿하게 보일 수 있으므로, srcset을 사용하여 2x, 3x 이미지를 제공하는 것이 좋습니다. 또한 Chrome 개발자 도구의 'Device Mode'를 활용하여 다양한 기기 환경을 시뮬레이션해보세요.

이미지 전략 : 차세대 포맷 & 반응형

> PROMPT INPUT

"Lighthouse 성능 점수를 개선하기 위해
이미지 최적화 태그를 작성해줘.

1. AVIF와 WebP 포맷을 우선 지원하고 JPG를 폴백으로 사용.
2. 화면 크기에 따른 반응형 이미지 (srcset/sizes) 적용.
3. 레이아웃 시프트(CLS) 방지를 위한 width/height 속성 명시."

💡 PRO TIP

loading="lazy" 속성은 뷰포트 아래에 있는 이미지에만 적용하세요.

LCP(Largest Contentful Paint) 요소인 히어로 이미지에 loading="eager"나 fetchpriority="high"를 사용하는 것이 좋습니다.

</> GENERATED CODE (HTML)

```
<!-- Picture 태그를 활용한 아트 디렉션 및 포맷 최적화 -->
<!-- 1순위: AVIF 포맷 (가장 높은 압축률) -->
<sourcesrcset src="img/hero.avif" type="image/avif" />

<!-- 2순위: WebP 포맷 (대부분의 모던 브라우저 지원) -->
<sourcesrcset src="img/hero.webp" type="image/webp"/>

<!-- 폴백 & 반응형 설정 (width/height 필수 for CLS 방지) -->

  srcset="img/hero-480.jpg 480w, img/hero-800.jpg 800w, img/hero-1200.jpg 1200w"
  sizes="(max-width: 600px) 480px, (max-width: 900px) 800px, 1200px"
  width="1200" height="675"
  loading="lazy" decoding="async" />
</picture>
```

CSS Minify & Purge

[P6] 최적화 2단계: 불필요한 CSS 제거 및 파일 압축

1



도구 설치

PurgeCSS 및 CLI 설치
개발 의존성으로 추가

```
→ npm install -D  
purgecss
```

2



Purge 실행

사용하지 않는 CSS 제거
콘텐츠 파일 스캔

```
→ npx purgecss  
--css style.css  
--content index.html  
--output dist/
```

3



Minify (압축)

공백/주석 제거로
파일 크기 최소화

```
npx esbuild dist/style.css --minify --  
outfile=dist/style.min.css
```

4



검증 및 예외처리

동적 클래스 유지 (safelist)
최종 크기 확인

```
→ ls -lh dist/  
style.min.css (4KB)  
# safelist: active, show
```

esbuild Optimization

번들링, 압축, 코드 분할 프로세스

1



번들링 & 압축

단일 파일 병합 및
공백/주석 제거



```
→ npx esbuild main.js  
--bundle --minify  
--outfile=dist/app.js
```

2



코드 분할

비동기 로딩을 위한
청크(Chunk) 파일 생성



```
→ --splitting  
--format=esm  
--outdir=dist/chunks
```

3



Tree Shaking

사용하지 않는 코드
자동 제거 (기본 활성화)



```
# Tree-shaking is  
# enabled by default  
# when bundling
```

4



소스맵 생성

디버깅을 위한 원본 코드
매핑 파일 생성



```
→ --sourcemap  
# Creates app.js.map  
# linked in output
```

OPTIMIZATION STRATEGY

[P6] 최적화4: Preload/Prefetch

>_ PROMPT INPUT

"초기 로딩 속도(LCP)를 개선하기 위해
핵심 리소스(폰트, 히어로 이미지)
를 높은 우선순위로 로드하는 HTML 태그를
작성해줘.

추가로, 사용자가 다음에 방문할 확률이 높은
페이지의 스크립트는 미리 받아오도록
(Prefetch) 설정해줘."

! CAUTION

Preload는 브라우저에게 "이건 당장 필요해!"라고
강제하는 것입니다. 너무 많이 사용하면 오히려 다른 중요한
리소스의 로딩을 방해할 수 있습니다.
Above the fold
리소스에만 신중하게 사용하세요.

</> GENERATED CODE (INDEX.HTML HEAD)

```
index.html

<head>
  <!-- 1. Preload: 현재 페이지의 핵심 리소스 (높은 우선순위) -->
  <!-- 폰트: 텍스트가 늦게 뜨거나 깜빡이는 현상(FOIT) 방지 -->
  <link rel="preload" href="/fonts/preload.woff2"
        as="font" type="font/woff2" crossorigin >

  <!-- 히어로 이미지: LCP(Largest Contentful Paint) 개선 -->
  <link rel="preload" href="/images/hero-banner.avif" as="image" >

  <!-- 2. Prefetch: 향후 네비게이션용 리소스 (낮은 우선순위, 유류 시간) -->
  <!-- 사용자가 다음에 클릭할 가능성이 높은 페이지의 자원 -->
  <link rel="prefetch" href="/js/project-detail.bundle.js" >
</head>
```

ACCESSIBILITY QUICK WINS

최소한의 노력으로 최대 효과를
내는 5가지 접근성 개선 전략



01

대체 텍스트 (Alt Text)

모든 의미 있는 이미지에 alt 속성을 제공하고, 장식용 이미지는 빈 값(alt="")으로 설정하여 스크린 리더 경험을 개선하세요.

02

폼 레이블 (Form Labels)

모든 입력 필드(input)에 label 태그를 연결(for-id)하여 사용자가 입력 목적을 명확히 알 수 있도록 합니다.

03

색상 대비 (Contrast)

텍스트와 배경 간의 명도 대비를 4.5:1 이상 (WCAG AA 기준)으로 유지하여 저시력자도 쉽게 읽을 수 있게 하세요.

04

포커스 순서 (Focus Order)

키보드 탭(Tab) 이동 순서가 시각적 흐름과 일치하는지 확인하고, tabindex 오남용을 피하세요.

05

자동화 도구 활용 (Axe DevTools)

Axe 또는 Lighthouse A11y 검사를 통해 자동으로 감지되는 오류(전체의 약 30%)를 먼저 해결하고, 경고 항목을 순차적으로 개선하세요.

[P6] 최적화6: 보안 헤더 기본

>_ PROMPT INPUT



"Netlify 배포를 위해 **_headers** 파일
에 적용할 기본 보안 헤더 설정을 작성해줘.

CSP(Content Security Policy), 클릭재킹
방지, MIME 스니핑 방지 등을 포함해서 보안
등급을 A+로 높이고 싶어."

💡 PRO TIP

CSP 설정 시 Google Fonts, Unsplash 등 외부
리소스를 사용하는 경우 해당 도메인을 **style-src** 나 **img-src**
에 명시적으로 허용해야 차단되지 않습니다.

</> GENERATED CONFIGURATION (PUBLIC/_HEADERS)

_headers

```
# 모든 경로(*)에 대해 다음 헤더 적용 /*
# 1. 클릭재킹 방지 (iframe 임베딩 차단)
X-Frame-Options: DENY

# 2. 브라우저의 MIME 타입 스니핑 차단
X-Content-Type-Options: nosniff

# 3. XSS 보호 (구형 브라우저 지원)
X-XSS-Protection: 1; mode=block

# 4. 리퍼러 정책 (개인정보 보호 강화)
Referrer-Policy: strict-origin-when-cross-origin

# 5. 권한 정책 (카메라, 마이크 등 민감한 기능 비활성화)
Permissions-Policy: geolocation=(), camera=(), microphone=()

# 6. 콘텐츠 보안 정책 (CSP) - 핵심 보안 설정
Content-Security-Policy: style-src 'self' https: data:; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com;
```

빌드 실패 : 경로 및 대소문자 이슈

로컬 환경(Windows/Mac)에서는 잘 되는데 배포(Linux) 후 에러가 발생하는 대표적 원인

⚠ 증상 (Symptom)

배포된 사이트에서 이미지가 액박으로 뜨거나, 콘솔에 404 에러가 발생하며 페이지가 로드되지 않음.

- ✖ GET /assets/Logo.png 404
- ✖ Module not found: Error: Can't resolve './Header'

🔍 원인 (Root Cause)

OS의 차이 : Windows와 macOS는 파일명의 대소문자를 구분하지 않지만, 배포 서버(Linux)는 이를 엄격히 구분함.

Filesystem:

/src/assets/**logo.png**

Code Import:

import img from './**Logo.png**'

↳ **Windows: OK / Linux: Fail**

✅ 해결 (Solution)

파일명 규칙을 통일하고, Git이 대소문자 변경을 인식하도록 설정을 변경해야 함.

- # 1. Git 설정 변경
`$ git config core.ignorecase false`
- # 2. 파일명 변경 (임시 이름 정류)

PRO TIP 프로젝트 시작 시 파일명은 모두 소문자(kebab-case)로 통일하는 규칙을 정하면 이러한 문제를 사전에 예방할 수 있습니다.

API 요청 실패: CORS 에러

외부 API(날씨, 뉴스 등)를 JavaScript로 직접 호출할 때 발생하는 보안 정책 위반 이슈

⚠ 증상 (Symptom)

Fetch 요청이 실패하고, 개발자 도구 콘솔에 빨간색으로 CORS 관련 에러 메시지가 출력됨.

❌ Access to fetch at '...'

from origin 'http://localhost:5500' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

🔍 원인 (Root Cause)

보안 정책 위반: 브라우저는 보안상 스크립트가 **다른 출처(Origin)**의 리소스에 접근하는 것을 기본적으로 제한함.

Origin 1 (My Site):

http://localhost:5500

Origin 2 (API Server):

https://api.weather.com

↳ 서버가 허용 헤더를 안 보내면 차단!

✅ 해결 (Solution)

개발 중에는 프록시 서버를 경유하거나, 백엔드 서버에서 헤더를 추가하도록 설정해야 함.

1. 서버 측 해결 (Node.js)

```
res.setHeader('Access-Control-Allow-Origin', '*');
```

2. 클라이언트 측 우회 (Proxy)

```
fetch('https://cors-anywhere.herokuapp.com/https://api.xyz')
```

PRO TIP 포트폴리오용 프론트엔드 프로젝트라면 **CORS Anywhere** 같은 공개 프록시를 임시로 사용하거나, Next.js/Vite의 **Proxy 설정**을 활용하세요.

CLS(레이아웃 시프트) 높음

페이지 로딩 중 요소들이 덜컥거리며 위치가 이동하는 현상 해결하기



증상 (Symptom)

이미지나 웹폰트가 늦게 로드되면서, 이미 있던 텍스트가 아래로 밀려나거나 레이아웃이 깨지는 현상.



Image Loading...

Lighthouse CLS Score: 0.25 (Poor)



원인 (Root Cause)

브라우저가 리소스 로드 전까지 크기를 몰라 공간을 확보하지 않음 (이미지) 또는 폰트 교체 시 깜빡임 (FOIT).

Missing Dimensions:

```

```

Font Loading:

```
font-display: block; /* Default */
```



해결 (Solution)

이미지/비디오의 width, height를 명시하여 공간을 예약하고, 폰트는 swap 옵션을 사용.

/* 1. 이미지 공간 예약 */

```

```

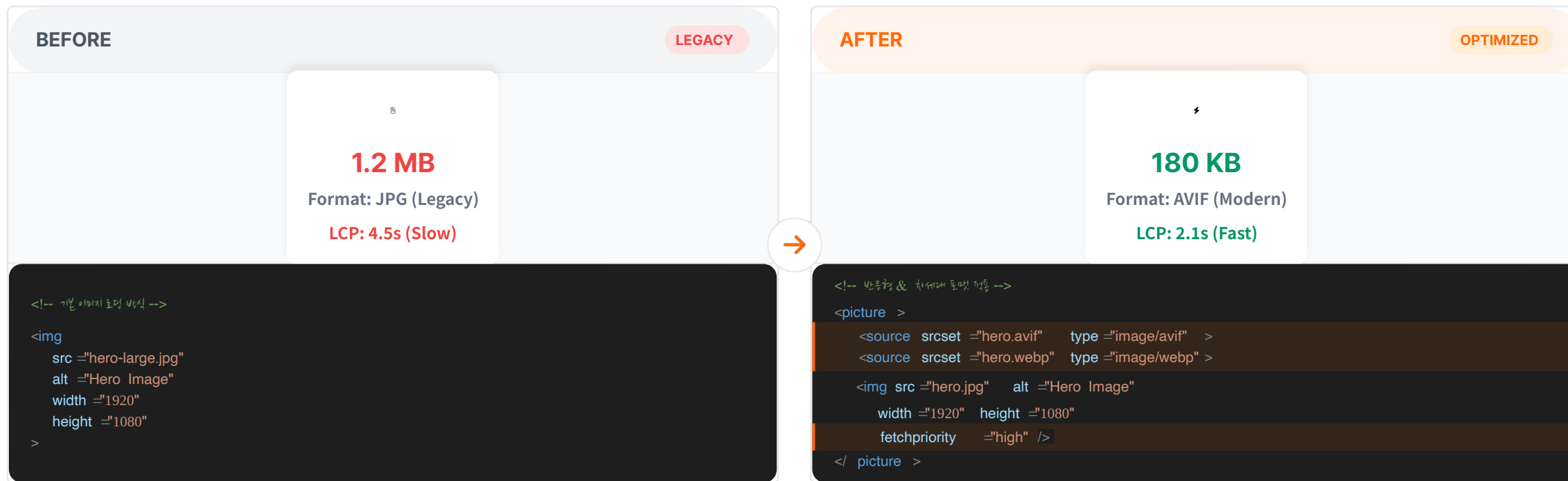
/* 2. 폰트 로딩 최적화 */

```
font-display: swap;
```

PRO TIP 반응형 이미지의 경우 **aspect-ratio** CSS 속성을 사용하면 width/height 속성 없이도 비율에 맞춰 공간을 미리 확보할 수 있습니다.

이미지 최적화 전/후 비교

원본 JPG를 차세대 포맷(AVIF)과 우선순위 힌트로 최적화하여 LCP 성능 개선



PRO TIP: LCP(Largest Contentful Paint) 최적화

페이지의 가장 큰 콘텐츠(LCP)가 되는 히어로 이미지는 `loading="lazy"`를 사용하면 안 됩니다. 대신 `fetchpriority="high"`를 추가하여 브라우저가 다른 리소스보다 우선적으로 다운로드하게 하세요.

DEPLOY PIPELINE



케이스 스터디 5: 배포 자동화

CASE STUDY 05

BEFORE

수동 빌드 및 배포 반복

로컬 환경 차이로 인한 빌드 실패 및 지연 발생

SOLUTION

GitHub Actions 워크플로우 도입

main 브랜치 push 시 자동 빌드 및 Pages 배포 트리거

BENEFIT

반복성 및 추적성 확보

모든 배포 과정이 로그로 기록되며 일관된 환경 보장

RESULT

안정성 향상 및 즉시 롤백

문제 발생 시 이전 커밋으로 즉시 복구 가능한 체계 구축

SECTION 04

DEV ENV

VS Code, Git, Node.js 설치부터
API 키 발급 및 환경 변수(.env) 설정까지.
바이브코딩을 위한 완벽한 개발 환경 구축.



SETUP

VS Code Installation

개발 환경 설정 프로세스 (Windows / macOS)



ESSENTIAL EXTENSIONS

01 Live Server

로컬 개발 서버를 실행하여 파일 저장 시 브라우저를 자동으로 새로고침합니다.
(실시간 미리보기)

02 Prettier - Code formatter

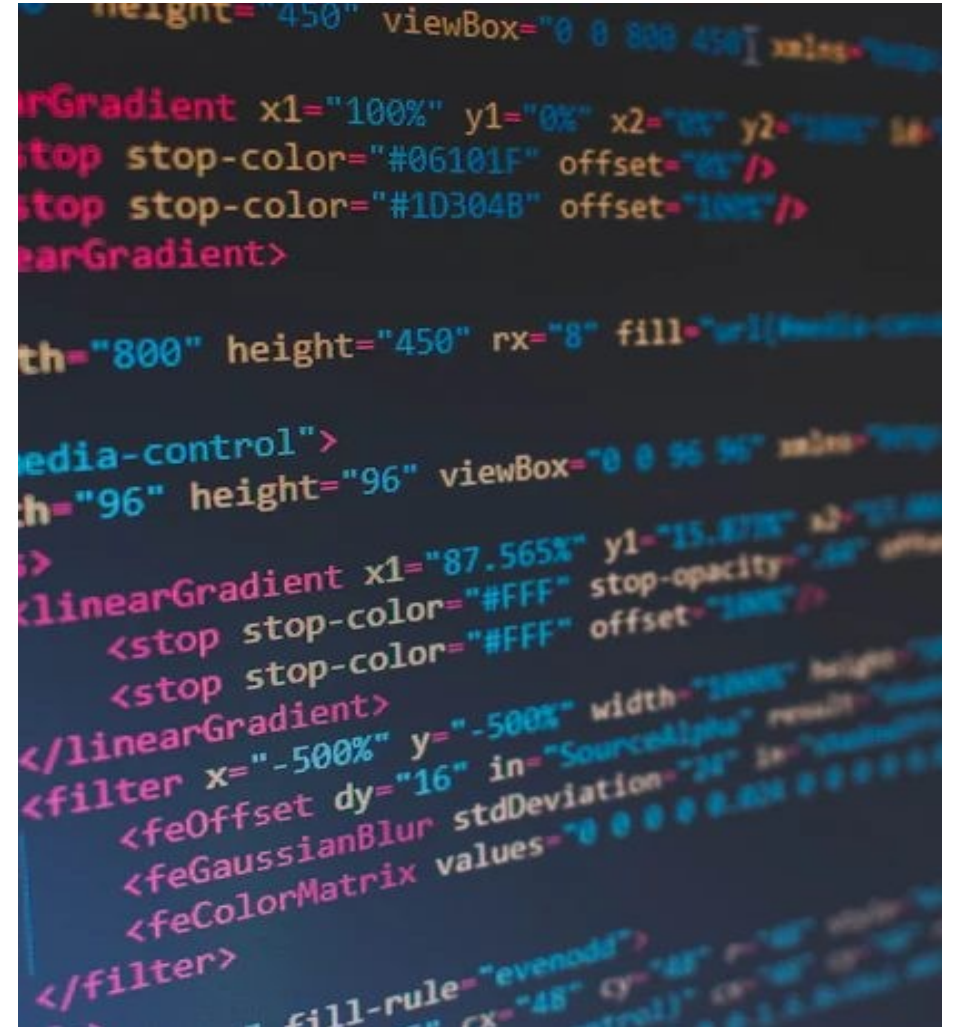
코드를 저장할 때마다 들여쓰기, 줄바꿈 등 스타일을 자동으로 정렬하여 가독성을 높입니다.

03 Auto Rename Tag

HTML 태그의 이름을 변경하면 짝이 되는 닫는 태그도 자동으로 함께 수정해줍니다.

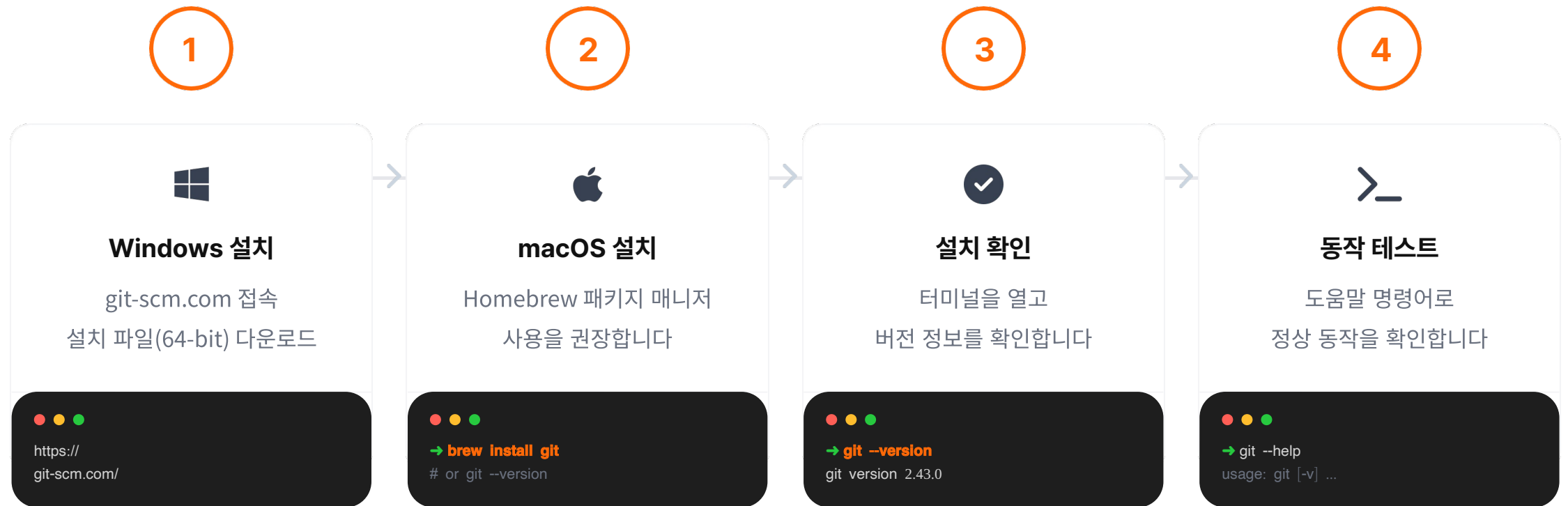
04 ESLint (선택)

JavaScript 코드의 문법 오류를 사전에 감지하고 일관된 코딩 규칙을 준수하도록 돕습니다.



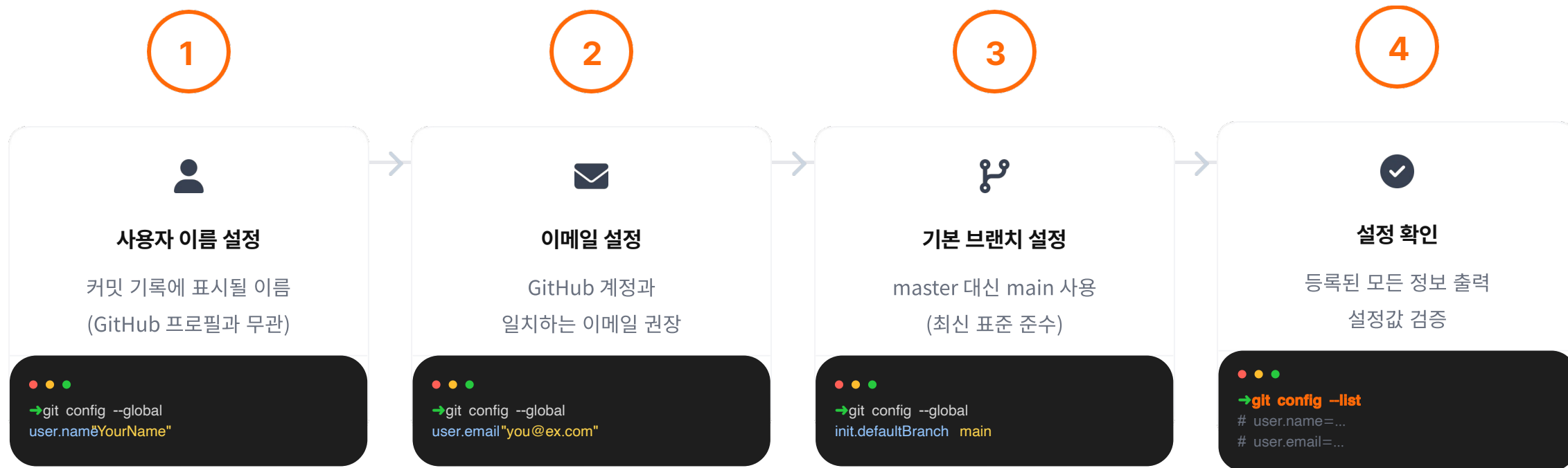
Git Installation

운영체제별 설치 가이드 (Windows / macOS)



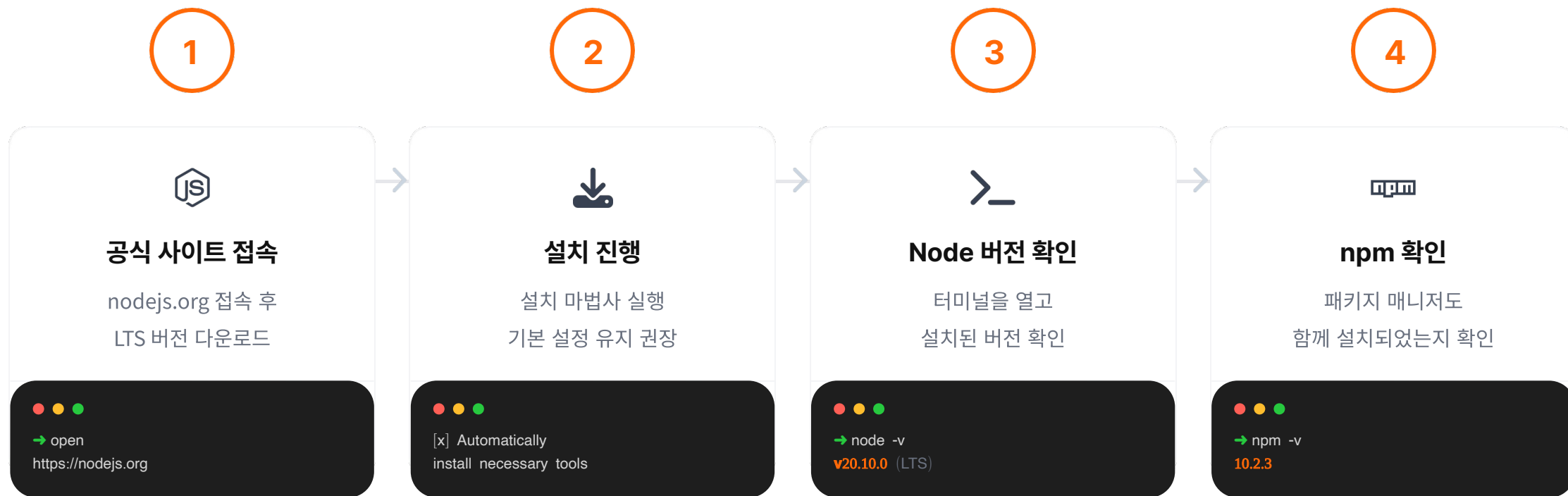
Git Configuration

사용자 정보 등록 및 초기 설정 명령어

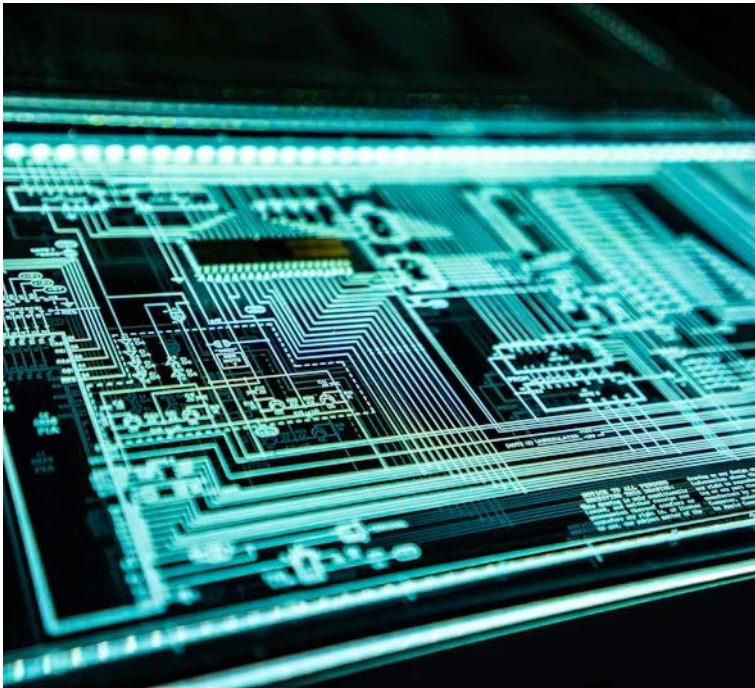


Node.js LTS Setup

안정적인 개발 환경 구축 (Windows / macOS)



ACCOUNT CREATION



Claude & Gemini 계정 준비



Claude 계정 (claude.ai)

Google 계정 또는 이메일로 가입하세요. 일부 기능 사용을 위해 전화번호 인증이 필요할 수 있습니다.



Gemini 계정 (gemini.google.com)

별도 가입 없이 기존 Google 계정으로 즉시 로그인하여 서비스를 이용할 수 있습니다.



정책 및 요금제 확인

무료 티어와 유료(Pro/Advanced) 모델의 차이점 및 API 사용량 제한 정책을 미리 확인하세요.

04

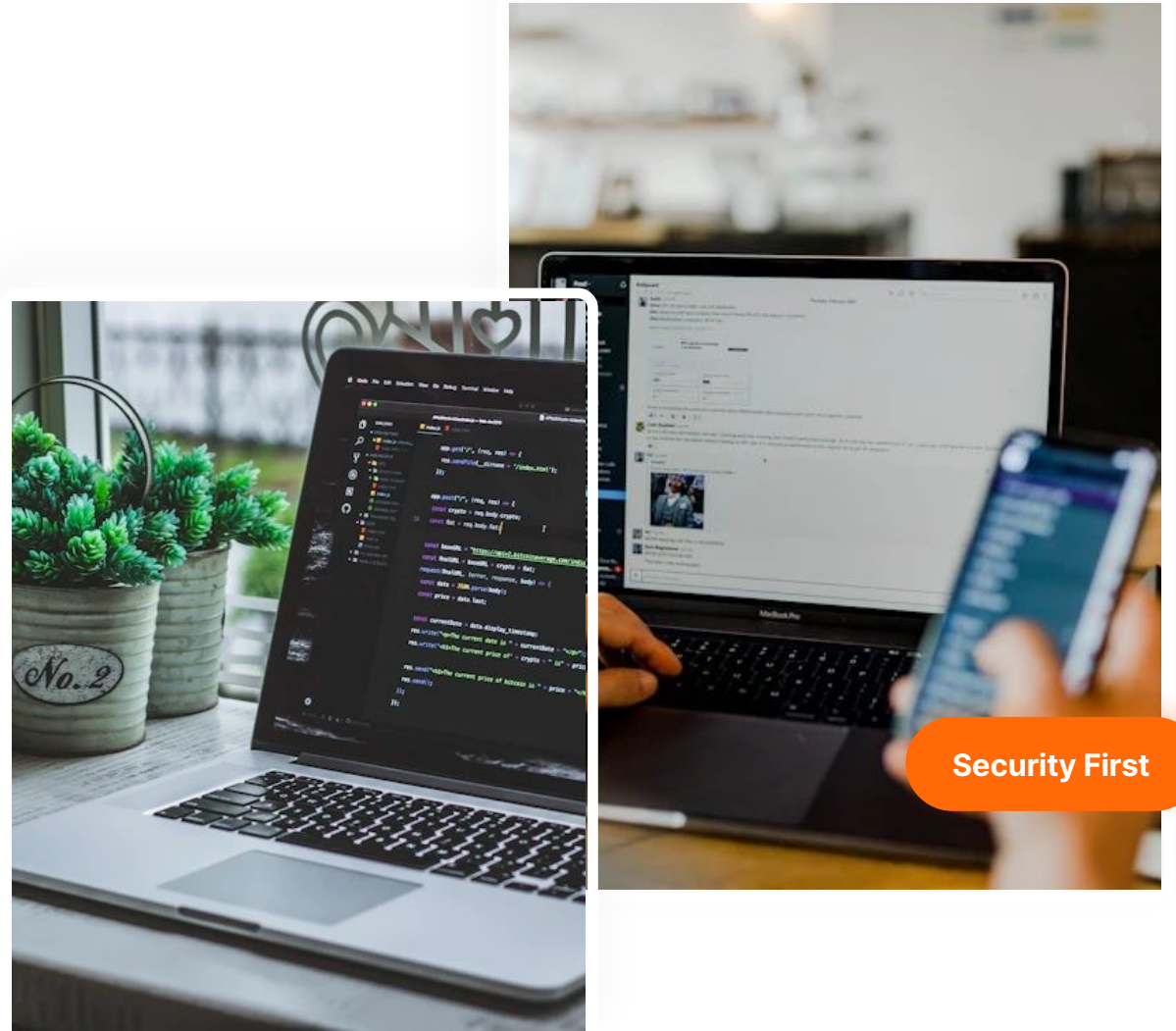
API 키 발급 및 주의사항

"API 키는 AI 서비스 사용을 위한
디지털 신분증이자 결제 수단입니다."

Claude 및 Gemini 개발자 콘솔에서 API 키를 발급받아야 합니다.
이 키는 외부에 절대 노출되어서는 안 됩니다.

- 🔑 Claude: console.anthropic.com 접속
- 📌 Gemini: aistudio.google.com 접속
- 🛡️ 주의: 키 노출 금지, GitHub 업로드 시 제외 필수

SETUP ENVIRONMENT



CONFIGURATION

.env 환경 변수 설정

>_ PROMPT INPUT



"API 키를 하드코딩하지 않고 안전하게 관리하기 위해 **.env 파일**을 생성하려고 해.

Claude API 키와 Gemini API 키를 환경 변수로 정의하는 예시 코드를 보여주고, 보안 주의사항도 함께 알려줘."

SECURITY ALERT

.gitignore 파일에 반드시 **.env**를 추가하세요. 실수로 API 키가 포함된 파일을 GitHub에 커밋하면 키가 즉시 노출되어 해킹의 위험이 있습니다.

GENERATED FILE (.ENV)

```
.env

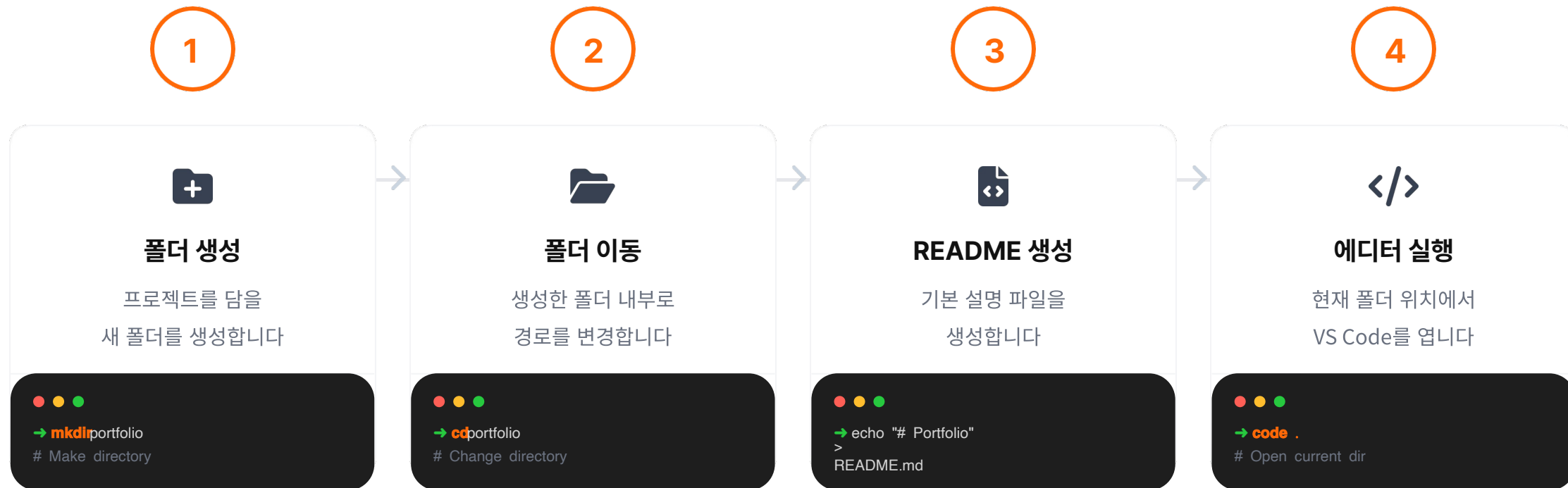
# 프로젝트 환경 변수 설정 파일
# 경고: 이 파일은 절대 GitHub 저장소에 커밋하지 마세요!
# -----
# Anthropic Claude API Configuration
# -----
CLAUDE_API_KEY=sk-ant-api03-kji29sdkf...

# -----
# Google Gemini API Configuration
# -----
GEMINI_API_KEY=AlzaSyD-283jd92...

# -----
# Optional Settings
# -----
NODE_ENV=developmentPORT=5500
# .gitignore 파일에 아래 줄을 꼭 추가하세요:
# .env
```

Project Folder Preparation

터미널 명령어로 프로젝트 시작하기



LIVE SERVER PREVIEW

01 HTML 파일 준비

프로젝트 루트 폴더에 index.html 파일을 생성하고 기본 구조(! + Tab)를 작성하여 준비합니다.

02 서버 실행 (Go Live)

VS Code 우측 하단의 'Go Live' 버튼을 클릭하거나, 파일에서 우클릭 후 'Open with Live Server'를 선택합니다.

03 로컬 호스트 접속 확인

브라우저가 자동으로 열리며 `http://127.0.0.1:5500/` 주소로 접속되는지 확인합니다.

04 실시간 리로드 테스트

코드 수정 후 저장(Ctrl+S) 시 브라우저 새로고침 없이 변경 사항이 즉시 반영되는지 검증합니다.



Git 초기화 및 커밋

첫 커밋 테스트

WORKFLOW STEPS

- 1 **git init**
현재 폴더를 Git 저장소로 초기화합니다. 숨겨진 .git 폴더가 생성됩니다.
- 2 **git add .**
작업 디렉토리의 모든 변경사항을 스테이징 영역(Staging Area)에 추가합니다.
- 3 **git commit -m "메시지"**
스테이징된 파일들을 확정하여 첫 번째 버전을 기록합니다.

CHECK POINT

원격 저장소(GitHub)에 연결하기 전,
반드시 로컬에서 첫 커밋이 완료되어야 에러 없이 푸시할 수 있습니다.

TERMINAL OUTPUT

```
bash — portfolio

user@mac: ~/portfolio$ git init
Initialized empty Git repository in /Users/user/portfolio/.git/

user@mac: ~/portfolio(main) $ git add .

user@mac: ~/portfolio(main) $ git status
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.html
    new file:   style.css

user@mac: ~/portfolio(main) $ git commit -m "chore: init project"
[main (root-commit) 8f3a1b2] chore: init project
2 files changed, 85 insertions(+)
create mode 100644 index.html
create mode 100644 style.css
```

VIBE CODING



AI와의 대화로 요구사항부터 구현, 검증까지.
프롬프트 전략과 대화 루프를 통해 코딩의 흐름을 바꿉니다.

BASICS

05 바이브코딩이란? 대화형 개발 방식

VIBE CODING BASICS

"AI와의 끊임없는 티키타카로
생각의 속도만큼 빠르게 개발합니다."

바이브코딩(Vibe Coding)은 단순히 코드를 생성하는 것을 넘어, AI를 동료 개발자로 대우하며 요구사항 정의부터 설계, 구현, 검증까지 대화로 풀어나가는 프로세스입니다.

- ✓ 요구사항 → 설계 → 구현 → 검증의 반복 루프
- ✓ 명확한 의도와 맥락(Context) 전달이 핵심
- ✓ 단순 복사-붙여넣기가 아닌 주도적인 리뷰 과정



TRADITIONAL VS VIBE CODING

비교 분석



PROCESS

직렬 프로세스 vs 반복적 순환

Linear vs Loop

FOCUS

문법과 디테일 vs 의도와 설계

Syntax vs Logic

ROLE

작성자(Writer) vs 감독관(Director)

Coder vs Reviewer

SPEED

타이핑 속도 vs 사고의 속도

Manual vs AI

프롬프트 작성법 : 명확성

AI가 의도를 정확히 파악하도록 돕는 3가지 핵심 요소



역할 (Role)

AI에게 수행해야 할 구체적인 전문가 페르소나를 부여하여 답변의 전문성, 깊이, 어조를 고정합니다.

EXAMPLE

"당신은 **10년차 시니어 UI 개발자**입니다. 웹 접근성(A11Y)을 최우선으로 고려해 주세요."



목표 (Goal)

해결하려는 문제와 최종적으로 얻고자 하는 결과물이 무엇인지 모호함 없이 명확하게 정의합니다.

EXAMPLE

"모바일 기기에서도 레이아웃이 깨지지 않는 **반응형 네비게이션 헤더**를 구현하는 것이 목표입니다."



제약 (Constraints)

사용할 기술 스택, 파일 구조, 금지된 패턴 등 작업의 구체적인 경계 조건을 명확히 나열합니다.

EXAMPLE

"Bootstrap 같은 프레임워크 없이, **순수 CSS (Vanilla CSS)**와 **BEM 명명법**만을 사용해 주세요."

프롬프트 작성법 : 맥락 제공

AI의 코드 품질을 높이는 3가지 필수 컨텍스트 정보



파일 구조 (Structure)

현재 프로젝트의 디렉토리 구조와 파일 위치를 명시하여 AI가 생성할 파일의 정확한 경로와 관계를 파악하게 합니다.

EXAMPLE

"현재 루트에는 **index.html**과 **/assets/css/style.css**가 있습니다. 이 구조에 맞춰 이미지 경로를 설정해주세요."



스타일 가이드 (Design)

색상 코드, 폰트, 여백 규칙 등 디자인 시스템 정보를 제공하여 일관된 룩앤필(Look & Feel)을 유지합니다.

EXAMPLE

"메인 컬러는 **#FF6908**이고, 모든 버튼은 border-radius: 30px입니다. 폰트는 Pretendard를 사용하세요."



참조 코드 (Reference)

기존에 작성된 코드 패턴이나 유사한 예시를 함께 전달하여 AI가 프로젝트의 코딩 컨벤션을 따르도록 유도합니다.

EXAMPLE

"아래 **Header 컴포넌트 코드 패턴**을 참고하여, 동일한 방식으로 Footer 영역을 마크업해주세요."

단계별 요청 Step-by-Step

복잡한 과제를 해결하는 체계적인 바이브코딩 접근법



1단계 : 구조와 골격

디자인이나 기능보다 논리적인 구조와 HTML 마크업을 먼저 정의하여 탄탄한 기초를 마련합니다.

PROMPT EXAMPLE

"스타일은 나중에 적용할 예정입니다. 먼저 포트폴리오의 **시맨틱한 HTML 구조만 작성**해주세요."



2단계 : 스타일링

확정된 HTML 구조 위에 CSS 스타일, 레이아웃, 배색을 입혀 시각적인 디자인을 완성합니다.

PROMPT EXAMPLE

"작성된 HTML을 바탕으로 **모던하고 깔끔한 CSS 스타일**을 적용해주세요. 테마 컬러는 오렌지입니다."



3단계 : 기능 및 최적화

반응형 대응, 자바스크립트 인터랙션, 웹 접근성 등 디테일한 고도화 작업을 수행합니다.

PROMPT EXAMPLE

"모바일에서도 잘 보이도록 **반응형 미디어 쿼리**를 추가하고, 이미지에 alt 태그를 점검해주세요."

코드 리뷰 및 리팩토링 요청

PROMPT INPUT

"현재 작성된 HTML 코드의 **성능(Performance)**과 **접근성(Accessibility)** 관점에서 리뷰해줘.

특히 다음 사항을 체크해줘:

- 시맨틱 태그의 적절한 사용 여부
- 이미지 alt 속성 누락 확인
- 불필요하게 중첩된 div 구조 개선

수정 제안 시, **변경 전/후 코드(Diff)**와 함께 **수정 이유**를 주석으로 명시해줘."

PRO TIP

단순히 "코드 고쳐줘"라고 하기보다 "가독성", "보안", "SEO" 등 구체적인 개선 목표(관점)를 제시하면 AI가 훨씬 전문적인 피드백을 제공합니다.

AI RESPONSE (REFACTORING SUGGESTION)

index.html (Refactored)

```
<!-- [Review] 전반적으로 양호하나, 시맨틱 구조와 접근성을 강화했습니다. -->

<body>
  <!-- [Refactor] header 내부에 nav를 배치하여 구조적 의미 강화 -->
  <headerclass="site-header" >
    <navaria-label="Main Navigation">
      <!-- [Fix] 누락된 alt 속성 추가 (스크린 리더 지원) -->
      <imgsrc="logo.png" alt="Stellaris Portfolio Logo" />
      <ul>... </ul>
    </nav>
  </header>

  <!-- [Refactor] 본문 콘텐츠는 div 대신 main 태그 사용 -->
  <mainid="main-content">
    <!-- [Optimize] 불필요한 div wrapper 2단계 제거 -->
    <sectionclass="hero-section" >
      <h1>Welcome to My Portfolio</h1>
    </section>
  </main>
</body>
```

EFFECTIVE DEBUGGING

디버깅 요청 - 에러 메시지 제공

>_ PROMPT INPUT

"모바일 메뉴 버튼을 클릭하면 브라우저 콘솔에 다음과 같은 에러가 발생해.

Uncaught TypeError: Cannot read properties of null (reading 'classList')

script.js 파일의 15번째 줄 toggleMenu 함수에서 발생했어. mobile-nav ID를 가진 요소를 못 찾는 것 같은데 해결책을 알려줘."

💡 DEBUGGING TIP

단순히 "안 돼요"라고 하기보다 에러 메시지(Stack Trace), 발생 상황(Steps to reproduce), 그리고 관련 코드 스니펫을 함께 제공하면 AI가 정확한 원인을 즉시 파악합니다.

</> AI SOLUTION & FIXED CODE

```
// 분석: DOM이 완전히 로드되기 전에 요소를 찾으려 하거나, ID가 오타일 수 있습니다.
// 해결: Null 체크(유효성 검사)를 추가하고 옵셔널 체이닝을 사용합니다.

function toggleMenu() {
  const menu = document.getElementById('mobile-nav');

  // 1. 요소가 존재하는지 먼저 확인합니다 (Safety Check)
  if (!menu) {
    console.error('Menu element not found! Check your HTML IDs');
    return;
  }

  // 2. 안전하게 클래스를 토글합니다
  menu.classList.toggle('active');
}

// 이벤트 리스너 추가 시 DOMContentLoaded 활용 권장
document.addEventListener('DOMContentLoaded', => { ... });
```

PERFORMANCE & QUALITY TIPS

AI 바이트코딩 결과물의 수준을 높이는 5가지 품질 관리 전략입니다.

01



작은 배치 (Small Batches)

한 번에 완벽한 코드를 기대하기보다, 기능 단위로 작게 나누어 생성하고 검증하는 것이 효율적입니다.

02



실시간 미리보기

Live Server 등을 활용해 코드 변경 사항을 즉시 시각적으로 확인하며 피드백 루프를 단축하세요.

03



체크리스트 기반 QA

구현 전 요구사항 목록을 작성하고, 하나씩 체크하며 누락된 기능이 없는지 체계적으로 검증하세요.

04



측정 및 로그 확인

성능 측정 도구와 브라우저 콘솔 로그를 주기적으로 확인하여 잠재적인 병목 구간을 조기에 식별하세요.

05



회귀 테스트 (Regression Test)

새로운 기능을 추가하거나 코드를 리팩토링할 때, 기존에 잘 작동하던 기능이 깨지지 않는지 지속적으로 확인해야 합니다.

안전/개인정보 가이드

AI 바이브코딩 시 반드시 지켜야 할
5가지 핵심 보안 수칙입니다.



01

🔑 API 키 관리 철저

API 키는 절대 코드에 하드코딩하지 말고, .env 환경 변수로 관리하며 저장소에 업로드하지 마세요.

02

🏠 민감 데이터 업로드 금지

고객의 개인정보(PII), 비밀번호, 금융 정보 등 민감한 데이터는 AI 프롬프트에 절대 입력하지 마세요.

03

© 저작권 및 라이선스 확인

생성된 코드의 라이선스 정책을 확인하고, 오픈소스 사용 시 저작권 침해 가능성을 검토하세요.

04

🛡️ 보안 취약점 검토

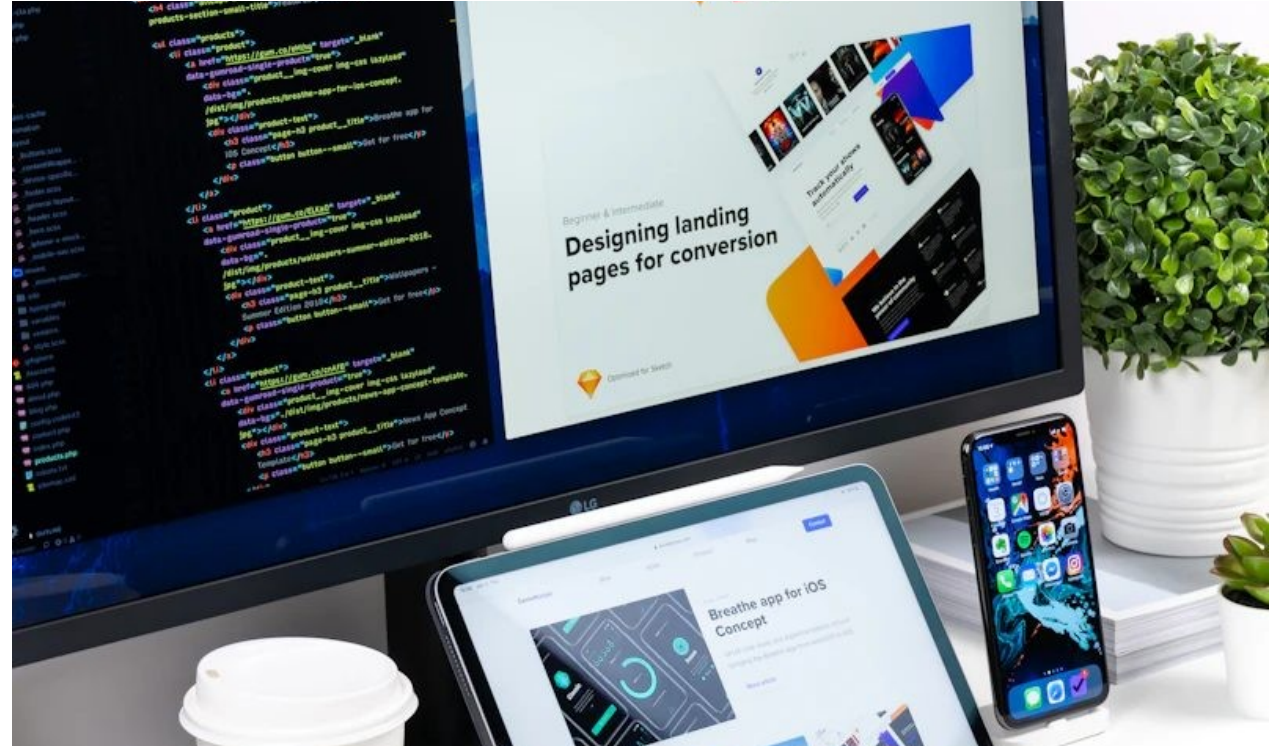
AI 생성 코드에는 XSS, SQL Injection 등 취약점이 있을 수 있으므로 반드시 보안 리뷰를 수행하세요.

05

🗑️ 데이터 비식별화 (De-identification)

테스트 데이터를 사용할 때는 실제 데이터 대신 가상의 더미 데이터를 사용하여 정보 유출 위험을 원천 차단하세요.

BUILD YOUR



구조 설계부터 시맨틱 HTML, 스타일링, 반응형 최적화까지.
AI와 함께 실전 포트폴리오 사이트를 단계별로 완성합니다.

PORTFOLIO

06폴더 구조 설계 프로젝트 기본 셋업

"체계적인 파일 구조는 AI가 프로젝트의 맥락을 정확히 이해하는 첫걸음입니다."

표준화된 웹 프로젝트 구조를 준수하면 유지보수가 쉬워지고, 협업 도구 및 AI 어시스턴트가 코드를 분석하기 훨씬 수월해집니다.

-  `root/` : 최상위 프로젝트 폴더 (portfolio)
-  `index.html` : 메인 진입 페이지
-  `assets/` : 이미지(img), 스타일(css), 스크립트(js)
-  `README.md` : 프로젝트 설명 및 배포 가이드
-  `.gitignore` : 버전 관리 제외 파일 설정

PROJECT STRUCTURE



HTML STRUCTURE PROMPT

HTML 기본 구조 생성

>_ PROMPT INPUT



"포트폴리오 웹사이트를 위한 **시맨틱한 HTML5 기본 골격**을 생성해줘.

header, main, footer 구조를 명확히 나누고, 검색 엔진 최적화(SEO)와 접근성(Accessibility)을 고려해 필요한 메타 태그와 속성을 모두 포함해줘."

💡 CHECK POINT

DOCTYPE 선언, **lang** 속성, 그리고 반응형을 위한 **viewport** 메타 태그가 포함되었는지 확인하세요.

</> GENERATED CODE (INDEX.HTML)

```
index.html

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>My Portfolio</title>
  <link rel="stylesheet" href="/style.css"/>
</head>
<body>
  <!-- Header: 로고 및 네비게이션 -->
  <header class="site-header">
    <nav aria-label="메인 네비게이션"><!-- Nav Links --></nav>
  </header>

  <!-- Main: 주요 콘텐츠 영역 -->
  <main id="main-content">
    <section id="hero"><!-- Hero Section --></section>
    <section id="projects"><!-- Projects List --></section>
  </main>

  <!-- Footer: 저작권 및 소셜 링크 -->
  <footer class="site-footer">
    <p>© 2026 My Portfolio. All rights reserved.</p>
  </footer>
</body>
</html>
```

STEP 2: HTML STRUCTURE

결과물 : index.html 골격 코드 - Part 1 : HEAD & MAIN 구조

>_ PREVIOUS PROMPT

"시맨틱 태그(header, nav, main, footer 기능)를 사용하여 포트폴리오의 **전체 HTML 기본 골격**을 만들어줘.

웹 접근성을 고려한 ARIA 랜드마크 역할과 메타 태그 설정을 포함해야 해."

Part 2 안내

Part 2에서 FOOTER와 닫는 태그를 완성합니다.

✔ CHECK POINT

lang="ko" 속성과 viewport 메타 태그가 포함되었는지 확인하세요. 이는 검색엔진 최적화(SEO)와 모바일 반응형 디자인의 필수 시작점입니다.

</> GENERATED SKELETON (INDEX.HTML - PART 1)

```
index.html

<!DOCTYPE html>
<html lang="ko" >
<head>
  <meta charset="UTF-8"/>
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title> My AI Portfolio </title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <!-- Header: 네비게이션 포함 -->
  <header class="site-header" >
    <nav aria-label="Main Navigation" >... </nav>
  </header>

  <!-- Main: 주요 콘텐츠 영역 -->
  <main id="main-content" >
    <section id="about" aria-labelledby="about-h" >... </section>
    <section id="projects" aria-labelledby="proj-h" >... </section>
    <section id="contact" aria-labelledby="contact-h" >... </section>
  <!-- Part 2에서 계속... -->
</body>
```

STEP 2: HTML STRUCTURE

결과물: index.html 골격 코드 - Part 2: FOOTER & CLOSING

PREVIOUS CONTEXT

Part 1 완료 항목:

- DOCTYPE 및 html/head 선언
- Header 및 Navigation 구조
- Main 및 주요 Section(About, Projects)

골격

→ CURRENT TASK (PART 2)

Part 2 목표:

이제 Contact 섹션 이후의 **Footer 영역**과
문서 닫는 태그를 완성하여 HTML 구조를
마무리합니다.

CHECK POINT

모든 섹션 태그(<section>)와 메인(<main>), 바디
(<body>), html 태그가 올바르게 닫혔는지 확인하세요.

</> GENERATED CODE (INDEX.HTML - PART 2)

```
<!-- ... Part 1 코드(Main 섹션들)에 이어서 ... -->
<section id="contact" aria-labelledby="contact-h">
  <h2 id="contact-h">Contact</h2>
  <!-- 연락처 폼 영역 -->
  <form action="https://formspring.io/f/..." method="POST">
    ...
  </form>
</section>
</main> <!-- Footer: 저작권 및 소셜 링크 -->
<footer class="site-footer">
  <div class="container">
    <p>© 2026 My Portfolio. All rights reserved.</p>
    <ul class="social-links" aria-label="Social Media Links">
      <li><a href="https://github.com/..." target="_blank" aria-label="GitHub Profile">GitHub</a></li>
      <li><a href="https://linkedin.com/..." target="_blank" aria-label="LinkedIn Profile">LinkedIn</a></li>
    </ul>
  </div>
</footer> <!-- 스크립트 로드 -->
<script src="/script.js"></script>
</body></html>
```

6. 포트폴리오 프로젝트 시작

>_ PROMPT INPUT

"작성된 index.html 구조에 맞춰 **style.css**를 작성해줘. 다음 디자인 시스템 요구사항을 준수해야 해:

1. **:root** 변수로 색상(Primary, BG, Text), 폰트, 간격 토큰을 정의할 것.
2. **BEM 네이밍 규칙**(.block__element--modifier)을 철저히 따를 것.
3. 모든 폰트 사이즈는 **16px(1rem)**을 기준으로 상대 단위(rem)를 사용할 것.
4. 모던한 Reset CSS를 포함해줘."

💡 PRO TIP

브랜드 컬러(예: #FF6908)나 선호하는 폰트(Pretendard)를 구체적인 값으로 제공하면, 추가 수정 없이 바로 사용할 수 있는 코드를 얻을 수 있습니다.

프롬프트 : CSS 스타일링

</> GENERATED CODE (STYLE.CSS)

```
/* 1. Design Tokens & Variables */
:root {
  --primary-color : #FF6908;
  --text-dark : #1F2937;
  --text-light : #6B7280;
  --bg-white : #FFFFFF;
  --font-main : 'Pretendard', sans-serif;
  --spacing-base : 1rem /* 16px */;
}

/* 2. Reset & Base Styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  font-family: var(--font-main);
  color: var(--text-dark);
  line-height: 1.6;
}

/* 3. BEM Component: Project Card */
.project-card {
  background-color: var(--bg-white);
  border-radius: 0.5rem;
  padding: var(--spacing-base);
  transition: transform 0.2s ease;
}

.project-card__title {
  font-size: 1.25rem;
  color: var(--primary-color);
  margin-bottom: 0.5rem;
}
```

CSS 스타일링 결과

결과물: style.css - CSS 변수

> PROMPT INPUT



"프로젝트 전반에 사용할 **유틸리티 토큰(색상, 간격, 타이포그래피)**을 정의하는 CSS 변수 (:root)를 작성해줘.

BEM 네이밍 규칙을 고려하고, 기본 폰트 크기는 16px로 설정해서 유지보수가 쉽도록 해줘."

💡 PRO TIP

CSS 변수(Custom Properties)를 활용하면 다크 모드 구현이나 브랜드 컬러 변경 시 한 곳에서 일괄 수정이 가능해 효율적입니다.

</> GENERATED CODE (STYLE.CSS)

```
style.css

/* Global Variables & Design Tokens */
:root {
  /* Color Palette */
  --color-primary : #FF6908;
  --color-secondary : #1F2937;
  --color-bg-light : #FFFFFF;
  --color-text-main : #111827;
  --color-text-muted : #6B7280;

  /* Spacing Scale (8px Grid System) */
  --spacing-sm : 8px;
  --spacing-md : 16px;
  --spacing-lg : 32px;
  --spacing-xl : 64px;

  /* Typography */
  --font-base : 16px;
  --font-family-main : 'Pretendard', sans-serif;
  --line-height-base : 1.5;
}

/* Basic Reset & Body Style */
body {
  font-family : var(--font-family-main);
  font-size : var(--font-base);
  line-height : var(--line-height-base);
  color : var(--color-text-main);
  background-color : var(--color-bg-light);
  margin : 0;
}
```

Responsive Prompts

모바일 최적화 및 반응형 웹 구현을 위한 핵심 프롬프트 전략



모바일 우선 (Mobile First)

작은 화면을 기준으로 기본 스타일을 먼저 정의하고, 화면이 커짐에 따라 스타일을 확장하도록 요청합니다.

EXAMPLE

"CSS는 **모바일 뷰(320px~)**를 기본으로 **작성**하고, 태블릿(768px)과 데스크탑(1024px)은 미디어 쿼리(min-width)로 추가해주세요."



유동적 레이아웃 (Fluid)

고정 픽셀(px) 대신 상대 단위(%, rem, vw)를 사용하여 다양한 화면 크기에 자연스럽게 대응하도록 지시합니다.

EXAMPLE

"컨테이너 너비는 px 대신 **%, rem** 단위와 **Flexbox/Grid**를 사용하여 유동적으로 구성하고, 이미지는 max-width: 100%를 적용해주세요."



명확한 분기점 (Breakpoints)

타겟 디바이스에 맞춰 구체적인 미디어 쿼리 분기점과 해당 구간에서의 레이아웃 변화를 명시합니다.

EXAMPLE

"미디어 쿼리는 **768px**과 **1024px** 두 가지 **분기점**을 사용하고, 모바일 1열 → 태블릿 2열 → 데스크탑 3열 그리드로 변환해주세요."

CSS RESPONSIVE DESIGN

모바일 최적화 코드

> PROMPT INPUT

"포트폴리오 사이트가 작은 화면에서도 잘 보이도록 미디어 쿼리(Media Queries)를 추가하고 싶어.

특히 768px 이하 모바일 화면에서 그리드 레이아웃을 1열로 변경하고, 이미지가 화면을 벗어나지 않게 하는 CSS 코드를 작성해줘."

💡 CHECK POINT

meta viewport 태그가 HTML 헤더에 반드시 포함되어 있어야 미디어 쿼리가 모바일 기기에서 정상적으로 작동합니다.

</> GENERATED CODE (STYLE.CSS)

```
/* 1. 이미지 반응형 기본 설정 */
img, video {
  max-width : 100%;
  height : auto ;
  display : block ;
}

/* 2. 모바일 브레이크포인트 (768px 이하) */
/* 컨테이너 패딩 조정 */
.container {
  padding : 0 20px ;
  width : 100%;
}

/* 그리드 레이아웃: 1열로 변경 */
.project-grid {
  grid-template-columns : 1fr ;
  gap: 24px ;
}

/* 폰트 크기 조정 */
h1 { font-size : 2.5rem ; }
.nav-menu { display : none; } /* 모바일 메뉴로 전환 */
```

STEP 04: 구현

헤더/네비게이션 구현 코드

> PROMPT INPUT



"반응형 웹사이트를 위한
고정 헤더와 네비게이션 HTML 을 작성해줘."

1. 키보드 사용자를 위한
'본문 건너뛰기(Skip Link)' 필수
2. 현재 페이지를 나타내는 aria-current 속성
포함
3. 로고는 왼쪽, 메뉴는 오른쪽에 배치되는 구
조"

💡 ACCESSIBILITY TIP

'Skip to content' 링크는 평소에는 숨겨져 있다가 탭 (Tab) 키로 포커스될 때만 보이도록 CSS로 처리해야 합니다. 이는 스크린 리더 사용자에게 매우 중요합니다.

</> GENERATED CODE (INDEX.HTML)

```
index.html

<!-- 접근성: 본문 바로가기 링크 (Tab키 포커스 시 노출) -->
<a href="#main-content" class="skip-link" >본문으로 건너뛰기 </a>
<!-- 헤더: 스크롤 시 상단 고정 -->
<header class="site-header sticky" >
  <div class="container" >
    <!-- 로고 영역 -->
    <a href="/" class="logo" aria-label="홈으로 이동" >Dev.Portfolio </a>

    <!-- 메인 네비게이션 -->
    <nav aria-label="메인 메뉴" >
      <ul class="nav-list" >
        <li> <a href="#about" >소개 </a> </li>
        <li> <!-- 현재 활성화된 페이지 표시 -->
          <a href="#projects" aria-current="page" class="active" >프로젝트 </a> </li>
        <li> <a href="#contact" >연락처 </a> </li>
      </ul>
    </nav>
  </div>
</header>
```

SECTION 06: 포트폴리오 프로젝트

소개 섹션(About Me) 코드

> PROMPT INPUT



"포트폴리오의
'About Me' 섹션 HTML 코드 를 작성해줘.

1. 프로필 이미지, 자기소개 글,
기술 스택 리스트 포함
2. '연락하기' 버튼(CTA) 포함
3. 시맨틱 태그 사용 및 접근성(ARIA) 준수"

💡 DESIGN TIP

이미지와 텍스트가 나란히 배치되는 구조(Grid/Flex)를
미리 생각하며 마크업 계층을 잡으세요. 모바일에서는
수직으로 쌓이는 구조(Stack)로 전환되어야 합니다.

</> GENERATED CODE (INDEX.HTML)

```
index.html (About Section)

<!-- About Me Section --><section id="about" class="about section" aria-labelledby="about-heading">
  <div class="container about__container">
    <!-- Section Title -->
    <h2 id="about-heading" class="section-title">About Me </h2>

    <!-- Content Wrapper for Flex/Grid -->
    <div class="about__content">
      <!-- Profile Image -->
      <div class="about__image">
        
      </div>

      <!-- Bio Text & Skills -->
      <div class="about__text">
        <p class="about__description">안녕하세요, 사용자 경험을 최우선으로 생각하는 프론트엔드 개발자입니다. </p>
        <!-- Skills List -->
        <ul class="about__skills" aria-label="기술 스택">
          <li>HTML5</li>
          <li>CSS3</li>
          <li>JavaScript (ES6+)</li>
          <li>React</li>
        </ul>

        <!-- CTA Button -->
        <a href="#contact" class="btn btn--primary">Contact Me<span aria-hidden="true"></span></a>
      </div>
    </div>
  </div>
</section>
```

PROJECT IMPLEMENTATION

프로젝트 섹션: 그리드 레이아웃

> PROMPT INPUT



"프로젝트 포트폴리오 목록을 보여줄
CSS Grid 레이아웃 코드를 작성해줘.

반응형(모바일 1열, 데스크탑 3열)으로
동작해야 하며, 카드에 마우스를 올렸을 때
(Hover)와 키보드 포커스 접근성 스타일도
포함해줘."

💡 PRO TIP

`repeat(auto-fit, minmax(300px, 1fr))` 패턴을
사용하면 미디어 쿼리 없이도 자동으로 줄바꿈되는
유연한 그리드를 만들 수 있습니다.

</> GENERATED CODE (STYLE.CSS)

```
style.css

/* Projects Section Grid Layout */
.projects-grid {
  display : grid ;
  /* 반응형: 최소 300px 네비 유지, 공간 남으면 채우기 */
  grid-template-columns : repeat(auto-fit, minmax(300px, 1fr)) ;
  gap: 2rem;
  padding: 2rem 0;
} /* Project Card Style */
.project-card {
  background: var(--bg-card) ;
  border-radius : 12px;
  box-shadow 0 4px 6px rgba(0, 0, 0, 0.05) ;
  transition : transform 0.3s ease, box-shadow 0.3s ease ;
} /* Hover & Focus Interactions */
.project-card:hover , .project-card:focus-within {
  transform : translateY(-5px) ;
  box-shadow 0 10px 15px rgba(0, 0, 0, 0.1) ;
  /* 키보드 접근성을 위한 아웃라인 */
  outline : 2px solid var(--primary-color) ;
  outline-offset : 4px;
}
```

프로젝트 카드 디자인 코드

>_ PROMPT INPUT



"프로젝트 섹션에 들어갈 '프로젝트 카드'
HTML 컴포넌트를 만들어줘.

1. 썸네일 이미지(hover시 확대 효과)
2. 프로젝트 제목 및 설명
3. 사용 기술 태그 (HTML, CSS 등)
4. 데모 및 코드 링크 아이콘 (FontAwesome)
5. 아이콘 링크에는 반드시 **aria-label**을 포함해줘."

💡 PRO TIP

카드 전체를 감싸는 `<article>` 태그를 사용하면 의미론적으로 더 명확합니다. 이미지 로딩 실패를 대비해 alt 텍스트를 꼼꼼히 작성하세요.

</> GENERATED CODE (INDEX.HTML)

index.html

```
<!-- Project Card Component <article class="project-card" >
  <!-- Image Container for Hover Effect -->
  <divclass="card-image">
    <imgsrc="/assets/project1.jpg" alt="개인 포트폴리오 사이트 미리보기" loading="lazy" />
  </div>
  <divclass="card-content">
    <h3class="card-title" >Personal Portfolio</h3>
    <pclass="card-desc">바이브코딩으로 제작한 반응형 웹사이트입니다. </p>
    <!-- Tech Stack Tags -->
    <divclass="card-tags">
      <spanclass="tag">HTML</span><spanclass="tag">CSS3</span>
    </div>
    <!-- Links with Accessibility Attributes -->
    <divclass="card-links" >
      <a href="#" aria-label "포트폴리오 데모 페이지 열기" target="_blank"><i class="fas fa-external-link-alt"/></a>
      <a href="#" aria-label "포트폴리오 소스코드 보기" target="_blank"><i class="fab fa-github"/></a>
    </div>
  </div>
</article>
```

SKILLS SECTION DEMO

스킬 섹션 - 프로그레스 바 구현

>_ PROMPT INPUT



"포트폴리오의 'Skills' 섹션을 만들어줘. 각 기술(HTML, CSS, JS)의 숙련도를 나타내는 **프로그레스 바(Progress Bar)** UI가 필요해.

시각 장애인을 위해 ARIA 속성(aria-valuenow 등)을 꼭 포함하고, 접근성을 고려한 시맨틱 마크업으로 작성해줘."

💡 PRO TIP

프로그레스 바의 너비(width)는 인라인 스타일이나 CSS 변수로 제어하면, 추후 JavaScript로 동적인 애니메이션 (스크롤 시 차오르는 효과 등)을 추가하기 쉽습니다.

</> GENERATED CODE (INDEX.HTML)

```
index.html

<!-- Skills Section -->
<section id="skills" class="skills-section">
  <h2 class="section-title">Technical Skills</h2>
  <div class="skills-list">
    <!-- HTML5 Skill Item -->
    <div class="skill-item">
      <div class="skill-info">
        <span class="skill-name">HTML5</span>
        <span class="skill-percent">90%</span>
      </div>
      <!-- Progress Bar Track -->
      <div class="progress-track">
        <!-- Screen Reader Attributes -->
        <div class="progress-fill"
          role="progressbar"
          aria-valuenow="90"
          aria-valuemin="0"
          aria-valuemax="100"
          aria-label="HTML5 proficiency"
          style="width: 90%;">
        </div>
      </div>
    </div>
  </div>
</section>
```

PROJECT IMPLEMENTATION

연락처 폼 구현 및 검증

> PROMPT INPUT



"포트폴리오의 **연락처(Contact)** 섹션에 들어갈 폼 HTML을 작성해줘.

다음 기능적 요구사항을 반드시 포함해야 해:

1. **접근성**: 모든 입력 필드에 label 연결
2. **유효성 검사**: 이메일 형식 확인 및 필수 입력(required) 속성
3. **스팸 방지**: 숨겨진 허니팟(Honeypot) 필드 추가"

✓ VALIDATION TIP

HTML5의 required, type="email", pattern 속성을 활용하면 복잡한 JS 없이도 브라우저 자체 유효성 검사 기능을 사용할 수 있습니다.

</> GENERATED CODE (INDEX.HTML)

```
index.html

<!-- Contact Form Section -->
<form id="contact-form" class="contact-form" action="#" method="POST">
  <!-- Spam Protection: Honeypot Field (Hidden) -->
  <div style="display:none;" aria-hidden="true" >
    <label for="bot-field" >Don't fill this out </label>
    <input type="text" name="bot-field" id="bot-field" />
  </div>

  <!-- Name Input -->
  <div class="form-group">
    <label for="name">이름 </label>
    <input type="text" id="name" name="name" required />
  </div>

  <!-- Email Input with Validation -->
  <div class="form-group">
    <label for="email">이메일 </label>
    <input type="email" id="email" name="email" placeholder="example@email.com" required />
  </div>

  <button type="submit" class="submit-btn" >보내기 </button>
</form>
```

GITHUB PAGES



Git의 기본 개념부터 원격 저장소 연결, GitHub Pages 설정, 그리고 최종 배포 검증까지.
내 로컬 코드를 전 세계와 공유하는 방법을 배웁니다.

DEPLOY

07

Git 기본 개념 : 버전 관리 시스템

GITHUB DEPLOYMENT

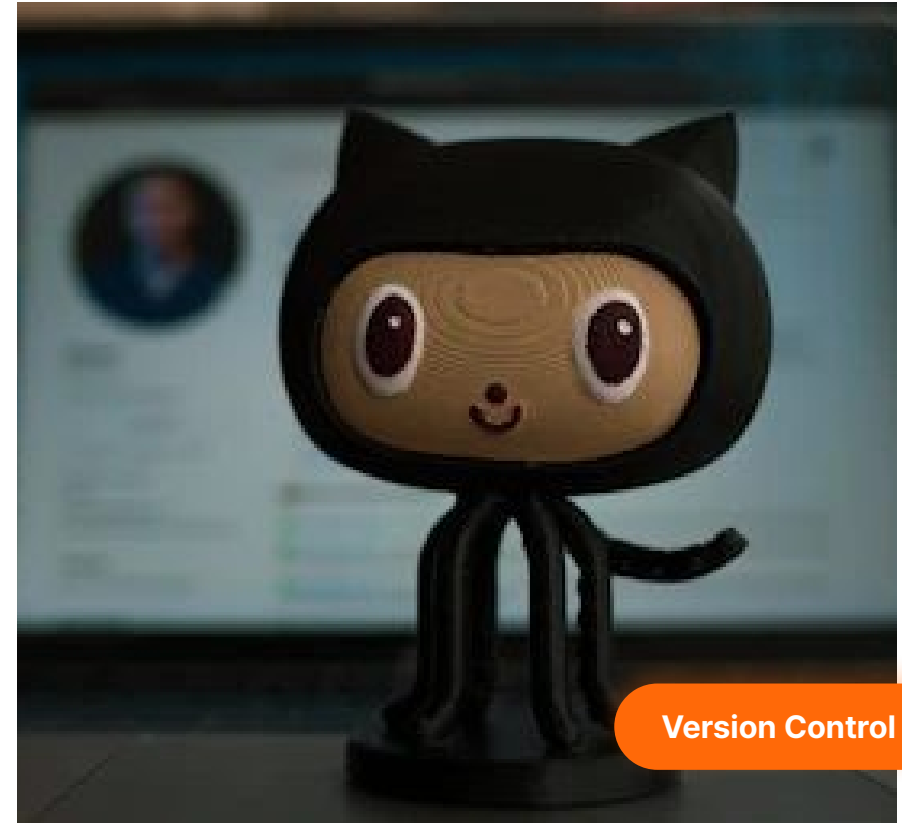
"프로젝트의 시간을 기록하고,
여러 사람과 안전하게 협업하는 도구"

Git은 단순한 파일 저장이 아닙니다.
프로젝트의 모든 변경 내역을 스냅샷으로 기록하여,
언제든 과거로 돌아가거나 새로운 기능을 실험할 수 있게 해줍니다.

📷 **스냅샷(Snapshot)** - 변경된 순간의 상태를 사진처럼 기록

🔗 **브랜치(Branch)** - 독립적인 병행 작업 공간 생성

📁 **워크플로우** - 수정 → 스테이징(Add) → 커밋(Commit)



GIT COMMANDS SUMMARY

01 저장소 초기화 및 상태 관리

git init 프로젝트 폴더를 Git 저장소로 초기화합니다.

git status 변경된 파일과 스테이징 상태를 확인합니다.

git add . 변경된 모든 파일을 스테이징 영역에 추가합니다.

02 버전 저장 및 이력 조회

git commit -m "msg" 스테이징된 파일을 스냅샷으로 저장합니다.

git log 프로젝트의 커밋 히스토리를 조회합니다.

03 원격 저장소 동기화 (GitHub)

git remote add 로컬 저장소를 GitHub 원격 저장소와 연결합니다.

git push 로컬의 커밋 내역을 GitHub로 업로드합니다.

git pull 원격 저장소의 최신 변경사항을 가져옵니다.

Git Cheat Sheet – Essential Commands



Setup & Config

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
git config --list
```

Configure user information and view settings.

Create & Clone Repos

```
git init
git clone <repo-url>
```

Initialize a local repository or download from a remote.

Basic Workflow

```
git status
git add .
git commit -m "message"
```

Check status, stage changes, and save a snapshot.

Branching

```
git branch
git branch <name>
git checkout <name>
git checkout -b <name>
```

List, create, switch, and create & switch branches.

Remote Repositories

```
git remote -v
git push origin main
git pull origin main
```

View remotes, send commits, and fetch changes.

Undo Changes

```
git reset --soft HEAD~1
git checkout -- <file>
git revert <commit>
```

Unstage, discard file changes, and reverse a commit.

Logs & History

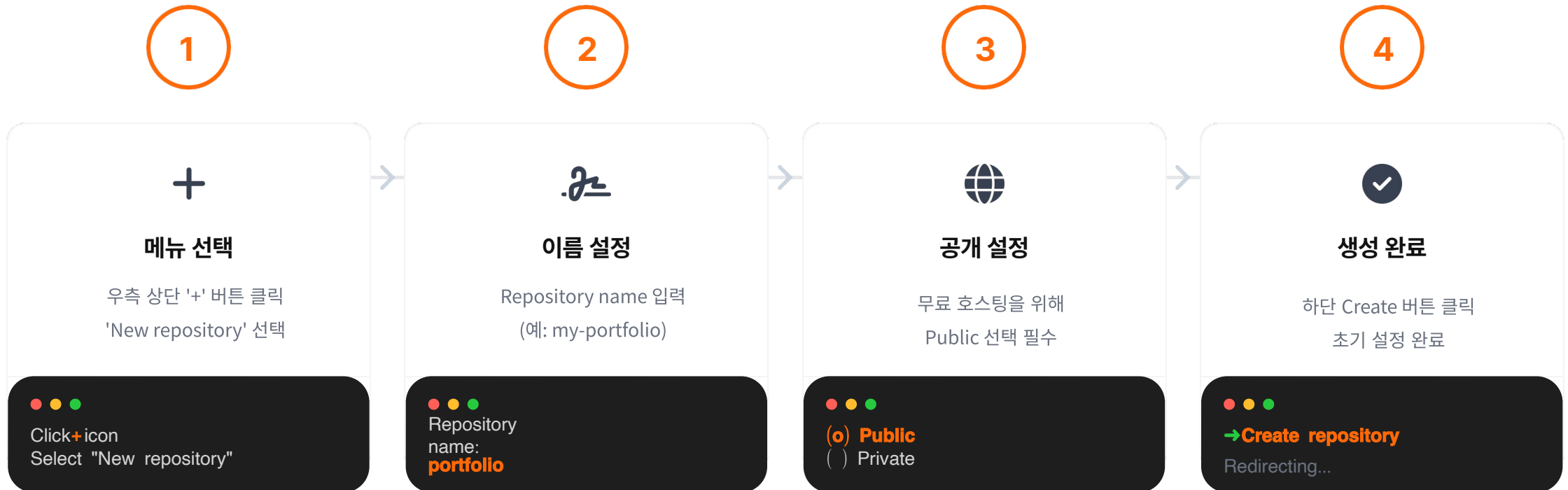
```
git log
git log --oneline --graph
```

View commit history, including condensed and graphical views.

© James Code Lab – Learn. Build. Share.

GitHub Repository

새 저장소 생성 단계별 가이드 (New repo → Settings)



GIT COMMANDS

로컬 저장소 초기화 및 커밋

>_ PROMPT INPUT



"작업 중인 포트폴리오 프로젝트 폴더를 Git
으로 관리하고 싶어."

로컬 저장소를 초기화하고, 작성한 모든 파일을
스테이징해서 첫 번째 커밋을 남기는 터미널
명령어를 알려줘."

! CHECK POINT

명령어를 입력하기 전에 `pwd`
또는 탐색기를 통해 현재 위치가 올바른
프로젝트 최상위 폴더 인지 반드시 확인하세요.

</> TERMINAL COMMANDS

bash

1. 현재 디렉토리를 **Git** 저장소로 초기화합니다

\$git init

Initialized empty Git repository in /Users/dev/portfolio/.git/

2. 모든 변경사항(파일)을 스테이징 영역(**Staging Area**)에 추가합니다

\$git add.

3. 설명 메시지와 함께 변경사항을 영구적으로 저장(**Commit**)합니다

\$git commit-m"feat: portfolio v1 initial commit"

[main (root-commit) a1b2c3d] feat: portfolio v1 initial commit

5 files changed, 145 insertions(+)

create mode 100644 index.html

create mode 100644 style.css

GITHUB DEPLOYMENT

원격 저장소 연결 및 코드 푸시

ACTION STEP



"이제 **GitHub에 생성한 빈 저장소**와 내 컴퓨터의 **로컬 프로젝트**를 연결할 차례입니다.

원격 저장소 주소를 등록하고, 작성한 코드를 업로드(Push)하여 배포 준비를 마칩니다."

⚠ CHECK POINT

GitHub 저장소 페이지에서 **HTTPS** 주소를 정확히 복사했는지 확인하세요. 주소 형식은 `https://github.com/사용자명/레포명.git` 입니다.

>_ TERMINAL COMMANDS

bash — portfolio

1. 원격 저장소 주소 등록 (**origin**이라는 별칭 사용)

```
git remote add origin https://github.com/USER/REPO.git
```

2. 브랜치 이름을 **main**으로 변경 (GitHub 기본 브랜치명)

```
git branch -M main
```

3. 코드를 원격 저장소로 업로드 (첫 푸시 시 **-u** 옵션 필수)

```
git push -u origin main
```

Enumerating objects: 15, done.

Counting objects: 100% (15/15), done.

Writing objects: 100% (15/15), 2.41 KiB | 1.20 MiB/s, done.

To https://github.com/USER/REPO.git

* [new branch] main -> main

GITHUB PAGES

SETUP

배포 설정 가이드

STEP 01 Settings 탭 이동

GitHub 저장소 상단 메뉴의 'Settings' 버튼을 클릭합니다.

STEP 02 Pages 메뉴 선택

좌측 사이드바 메뉴 중 'Code and automation' 아래의 'Pages'를 선택합니다.

STEP 03 Build & Deploy

Source를 'Deploy from a branch'로, Branch를 'main' / '/' (root)로 설정합니다.

STEP 04 Save & 확인

'Save' 클릭 후 상단에 생성되는 배포 URL(github.io)을 확인합니다.

DEPLOYMENT VERIFICATION

배포 후 성공적인 서비스를 위해
반드시 확인해야 할 체크리스트입니다.



01

URL 접속 확인

제공된 GitHub Pages
URL(username.github.io/repo)로 접속하여
페이지가 정상 로드되는지 확인하세요.

02

빌드 상태 체크

GitHub Repository의 Actions 탭에서 배포
워크플로우가 초록색(Success)인지 반드시
확인해야 합니다.

03

강력 새로고침

이전 버전 캐시를 무시하기 위해
Windows(Ctrl+F5) 또는
Mac(Cmd+Shift+R)으로 강력
새로고침하세요.

04

404 에러 대처

404 발생 시 Settings > Pages의 브랜치
설정을 확인하거나 루트 경로의 index.html
존재 여부를 체크하세요.

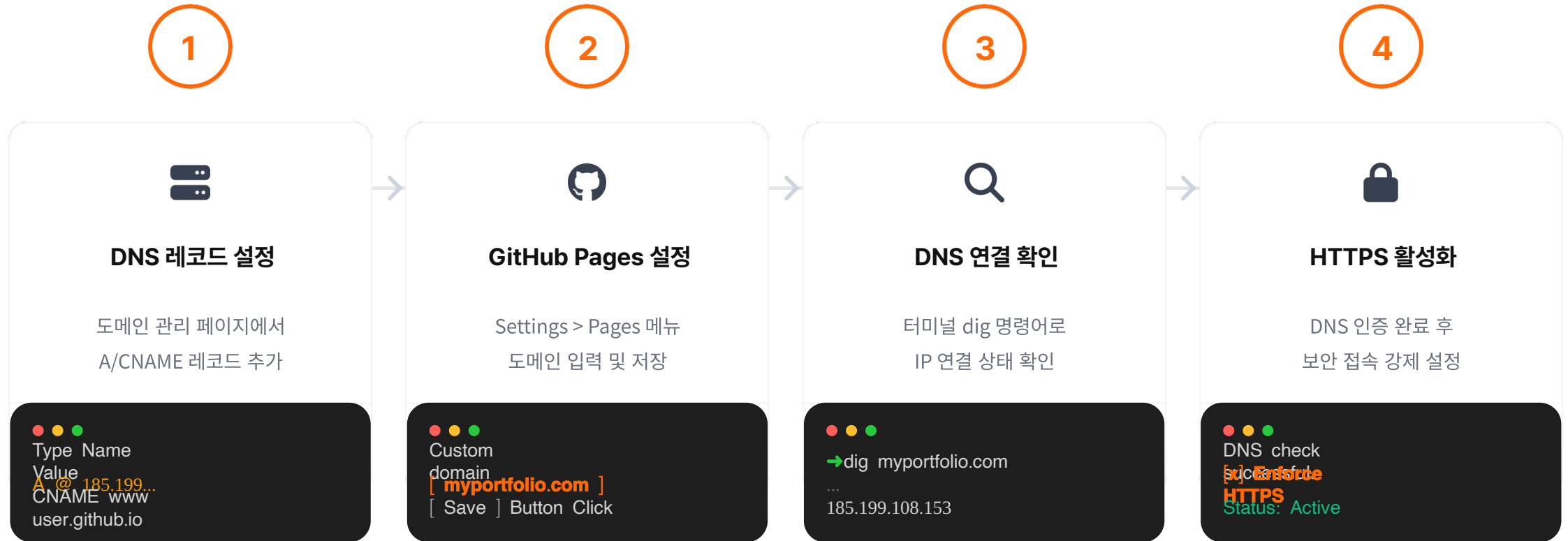
05

캐시 지연 인지 (Delay Awareness)

GitHub Pages는 배포 완료 후 실제 반영까지 1~10분 정도 소요될 수 있습니다.
설정이 올바르다면 당황하지 말고 잠시 기다린 후 다시 접속해보세요.

Custom Domain

커스텀 도메인 연결 및 HTTPS 설정 프로세스



AUTOMATED DEPLOYMENT

01 **gh-pages 패키지 활용**
npm 스크립트에 deploy 명령어를 추가하여, 터미널 명령어 하나로 빌드부터 gh-pages 브랜치 배포까지 한 번에 처리하는 방법입니다.

02 **GitHub Actions 워크플로우**
.github/workflows 폴더에 설정 파일을 추가하여, 코드를 푸시할 때마다 클라우드 상에서 자동으로 빌드와 배포가 수행되도록 설정합니다.

03 **PR 머지 시 자동 빌드**
팀 프로젝트 시 유용한 기능으로, Pull Request가 Main 브랜치에 병합(Merge)되는 순간 최신 버전이 라이브 서버에 즉시 반영됩니다.

Git Cheat Sheet – Essential Commands



Setup & Config

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
git config --list
```

Configure user information and view settings.

Create & Clone Repos

```
git init
git clone <repo-url>
```

Initialize a local repository or download from a remote.

Basic Workflow

```
git status
git add .
git commit -m "message"
```

Check status, stage changes, and save a snapshot.

Branching

```
git branch
git branch <name>
git checkout <name>
git checkout -b <name>
```

List, create, switch, and create & switch branches.

Remote Repositories

```
git remote -v
git push origin main
git pull origin main
```

View remotes, send commits, and fetch changes.

Undo Changes

```
git reset --soft HEAD~1
git checkout -- <file>
git revert <commit>
```

Unstage, discard file changes, and reverse a commit.

Logs & History

```
git log
git log --oneline --graph
```

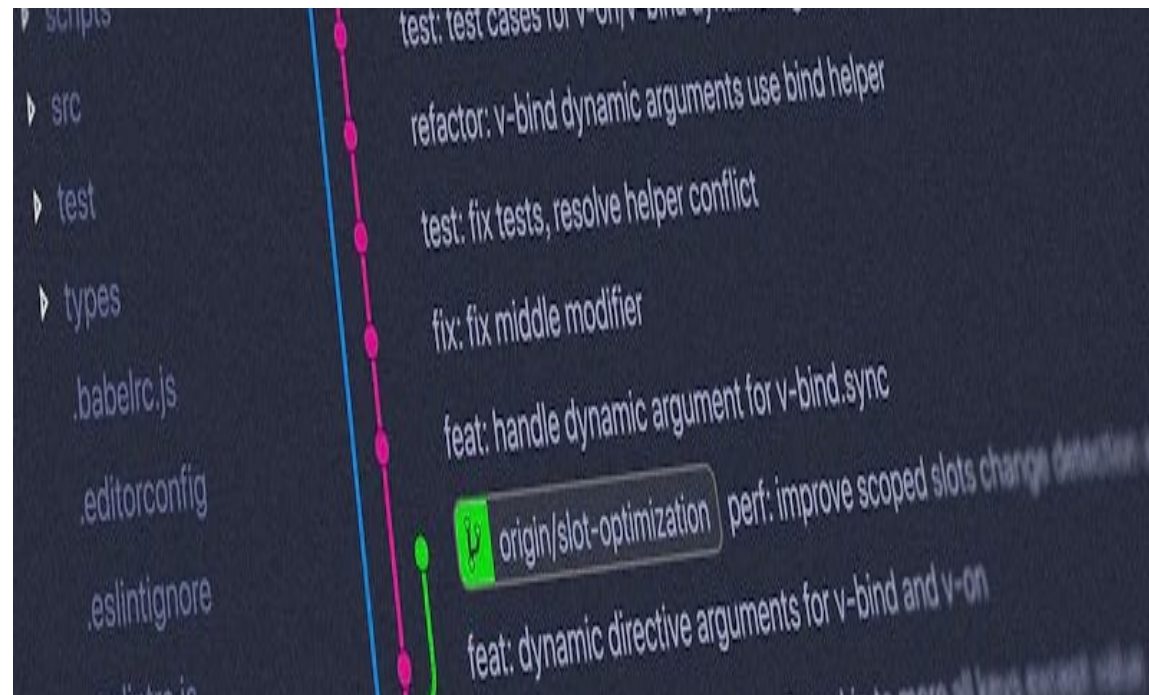
View commit history, including condensed and graphical views.

© James Code Lab – Learn. Build. Share.

SECTION 08

LEVEL UP

애니메이션, 인터랙션, 다크모드, 필터링,
그리고 SEO 최적화까지. 포트폴리오의 완성도를 전문가 수준으로 높입니다.



SKILLS

ADVANCED CSS

CSS 애니메이션 - Fade-in 효과

>_ PROMPT INPUT



"웹사이트 로딩 시 요소들이 부드럽게 나타나도록 Fade-in 애니메이션 CSS를 작성해줘."

재사용 가능한 유틸리티 클래스 형태로 만들고, 자연스러운 움직임을 위해 transform 속성을 함께 사용해줘."

💡 PERFORMANCE TIP

opacity와 transform

속성만 애니메이션화하면 브라우저의 리플로우(Reflow)를 방지하여 렌더링 성능을 최적화할 수 있습니다.

</> GENERATED CODE (STYLE.CSS)

```
style.css

/* 1. 애니메이션 키프레임 정의 */
@keyframes fadeIn {
  from {
    opacity : 0;
    transform : translateY (20px);
  }
  to {
    opacity : 1;
    transform : translateY (0);
  }
}

/* 2. 유틸리티 클래스 생성 */
.fade-in {
  opacity : 0; /* 로딩 전 감춤 방지 */
  animation : fadeIn 0.6s ease-out forwards;
}

/* 3. 순차적 등장 딜레이 (선택 사항) */
.delay-100 { animation-delay : 0.1s; }
.delay-200 { animation-delay : 0.2s; }
```

SCROLL INTERACTION

JS 스크롤 인터랙션 - IntersectionObserver

> PROMPT INPUT



"웹사이트의 모든 <section> 요소가 화면 뷰포트에 20% 정도 진입했을 때, 부드럽게 나타나는 애니메이션 효과를 주려고 해."

스크롤 이벤트 대신 성능이 좋은

IntersectionObserver API

를 사용해서, 요소가 보일 때 'visible' 클래스를 추가하는 JavaScript 코드를 작성해줘."

💡 PRO TIP

`observer.unobserve(entry.target)`
을 사용하면 애니메이션이 한 번만 실행되도록 하여 ‘불필요한 연산을 줄일 수 있습니다.’

</> GENERATED CODE (SCRIPT.JS)

```
script.js

// 1. 옵션 설정: 뷰포트의 20%가 보일 때 트리거
const options = {
  root: null, // null은 뷰포트를 의미
  rootMargin: '0px',
  threshold: 0.2
}; // 2. 콜백 함수: 교차 상태가 변경될 때 실행

const observerCallback = (entries, observer) => {
  entries.forEach(entry => {
    // 요소가 화면에 진입했는지 확인
    if (entry.isIntersecting) {
      entry.target.classList.add('visible');
      // 성능 최적화: 한 번 실행 후 관찰 중단
      observer.unobserve(entry.target);
    }
  });
}; // 3. Observer 인스턴스 생성 및 타겟 관찰 시작

const observer = new IntersectionObserver(observerCallback, options);
document.querySelectorAll('fade-in-section').forEach(section => {
  observer.observe(section);
});
```

다크모드: CSS 변수 설정

> PROMPT INPUT

"CSS 변수(Custom Properties)를 사용하여 **라이트 모드와 다크 모드** 테마를 관리하는 코드를 작성해줘.

:root에는 기본값(라이트)을 정의하고,
[data-theme='dark'] 속성 선택자를 이용해
다크 모드 값을 재정의하는 방식으로 구현해줘."

💡 PRO TIP

transition 속성을 body나 주요 컨테이너에 적용하면,
테마가 변경될 때 배경색과 글자색이 부드럽게 전환되어
훨씬 고급스러운 느낌을 줍니다.

</> GENERATED CODE (STYLE.CSS)

```
/* 1. 기본 테마 변수 정의 (Light Mode) */
:root {
  --bg-primary: #FFFFFF;
  --text-primary: #1F2937;
  --accent-color: #FF6908;
  --card-bg: #F3F4F6;
  --border-color: #E5E7E;
} /* 2. 다크 모드 재정의 (HTML 태그 속성 기준) */
[data-theme="dark"] {
  --bg-primary: #111827;
  --text-primary: #F9FAF;
  --accent-color: #FF8A4; /* 어두운 배경 대비 명도 조절 */
  --card-bg: #1F2937;
  --border-color: #374151;
} /* 3. 변수 적용 및 트랜지션 */
body {
  background-color: var(--bg-primary);
  color: var(--text-primary);
  transition: background-color 0.3s ease, color 0.3s ease;
}
```

JAVASCRIPT LOGIC

다크모드 토글 구현

> PROMPT INPUT

"사용자가 토글 버튼을 클릭하면 **HTML 루트 요소의 data-theme 속성**을 'dark' 또는 'light'로 변경하는 자바스크립트 코드를 작성해줘.

중요한 점은 **localStorage**를 사용하여 사용자의 선택을 저장하고, 페이지를 새로고침해도 설정이 유지되어야 해."

💡 UX TIP

처음 방문한 사용자에게는 prefers-color-scheme 미디어 쿼리를 사용하여 OS 시스템 설정(다크/라이트)을 자동으로 감지해 적용하는 것이 좋습니다.

</> GENERATED CODE (SCRIPT.JS)

```
// 1. 요소 선택
const toggleBtn = document.querySelector('#theme-toggle');
const root = document.documentElement;

// 2. 초기 테마 설정 (저장된 값 확인)
const savedTheme = localStorage.getItem('theme');
if (savedTheme) {

    root.setAttribute('data-theme', savedTheme);
}

// 3. 토글 이벤트 리스너
toggleBtn.addEventListener('click', () => {

    const currentTheme = root.getAttribute('data-theme');
    const newTheme = currentTheme === 'dark' ? 'light' : 'dark';

    // 테마 적용
    root.setAttribute('data-theme', newTheme);

    // 로컬 스토리지에 저장 (영구 보존)
    localStorage.setItem('theme', newTheme);
});
```

고급 기능 추가

프로젝트 필터링 - JavaScript

>_ PROMPT INPUT



"프로젝트 섹션에 카테고리별 필터링 기능을 추가하고 싶어.

'Web', 'App', 'Design' 버튼을 클릭하면,
해당 data-category 속성을 가진 프로젝트
카드만 남기고 나머지는 숨기는

JavaScript

코드를 작성해줘.

현재 선택된 버튼에는 'active' 클래스를
붙여줘."

💡 PRO TIP

DOM 조작을 최소화하기 위해 CSS 클래스(.hide)를
토글하는 방식을 사용하면 성능이 더 좋고, 페이드
애니메이션을 추가하기도 쉽습니다.

</> GENERATED CODE (SCRIPT.JS)

script.js

```
const filterBtns = document.querySelectorAll('.filter-btn');
const projects = document.querySelectorAll('.project-card');

filterBtns.forEach(btn => {
  btn.addEventListener('click', () => {
    // 1. 버튼 활성화 상태 업데이트
    document.querySelector('.filter-btn.active').classList.remove('active');
    btn.classList.add('active');

    // 2. 데이터 필터 값 가져오기
    const filterValue = btn.getAttribute('data-filter');

    // 3. 프로젝트 필터링 로직
    projects.forEach(card => {
      if (filterValue === 'all' || card.dataset.category === filterValue) {
        card.style.display = 'block';
      } else {
        card.style.display = 'none';
      }
    });
  });
});
```

SEO & QUALITY CHECKLIST



완성도 높은 포트폴리오를 위한 5가지 필수 점검 사항입니다.

01

🔍 메타 태그 & OG

검색 엔진 최적화(SEO)를 위한 메타 태그와, SNS 공유 시 보여질 오픈 그래프(OG) 정보를 올바르게 설정하세요.

02

✂ 이미지 최적화

이미지 용량을 압축하고 WebP 형식을 사용하여 로딩 속도를 개선하세요. 성능 점수에 큰 영향을 줍니다.

03

👤 접근성 (A11y)

이미지 대체 텍스트(alt), 시맨틱 태그, ARIA 속성을 사용하여 보조 기술 사용자도 웹사이트를 이용할 수 있게 하세요.

04

👁 컬러 콘트라스트

텍스트와 배경색 간의 명도 대비를 충분히 확보하여 가독성을 높이세요. 최소 4.5:1 비율을 권장합니다.

05

⌨ 키보드 내비게이션

마우스 없이 탭(Tab) 키만으로 모든 링크와 버튼에 접근하고 상호작용할 수 있는지 확인하세요. 포커스 스타일도 명확해야 합니다.

COURSE SUMMARY

01 AI 코딩 도구 마스터

Claude Code와 Gemini의 강점을 이해하고, 상황에 따라 적절한 도구를 선택하여 활용하는 능력을 갖췄습니다.

02 바이트코딩 프롬프트 전략

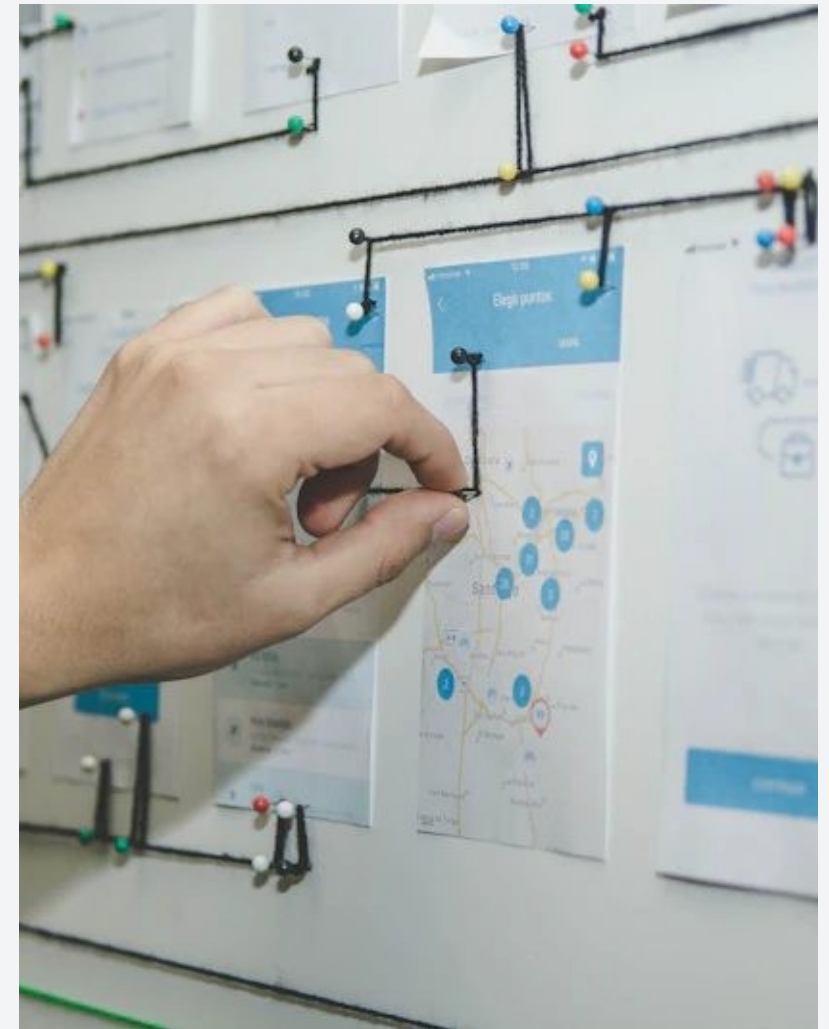
명확한 맥락과 제약 조건을 포함한 프롬프트로 AI와 협업하며 효율적으로 코드를 생성하는 방법을 익혔습니다.

03 포트폴리오 제작 및 배포

HTML/CSS로 반응형 구조를 설계하고 GitHub Pages를 통해 전 세계에 결과물을 배포했습니다.

04 고급 기능 확장

다크모드, 애니메이션, SEO 최적화 등 완성도를 높이는 추가 기능들을 적용하여 포트폴리오를 업그레이드했습니다.



09

추가 학습 리소스 & 링크

"배움은 끝이 없습니다.
공식 문서와 커뮤니티에서 계속 성장하세요."

수업 이후에도 지속적인 학습을 위해 엄선한 필수 리소스 모음입니다.
즐거찾기에 추가해두고 필요할 때마다 참고하세요.

-  공식 문서: Anthropic Docs, Google AI Studio
-  샘플 레포: GitHub Portfolio Template
-  가이드: MDN Web Docs, Web.dev 접근성 가이드

RESOURCES



Bookmarks

BEST PRACTICES CHECKLIST

성공적인 포트폴리오 배포와
유지보수를 위한 5가지 핵심
점검 사항입니다.



01

✂ 작은 이터레이션

거창한 계획보다 작은 기능 단위로 개발하고
배포하는 주기를 짧게 가져가 피드백을 빠르게
반영하세요.

02

🔗 철저한 버전 관리

의미 있는 커밋 메시지를 작성하고, 기능별 브랜치
전략을 활용하여 코드를 안전하게 관리하세요.

03

🖥 미리보기 및 테스트

배포 전 로컬 서버와 다양한 모바일 환경에서
충분히 테스트하여 UI 깨짐이나 오류를 방지하세요.

04

🕒 웹 접근성 준수

시맨틱 태그 사용과 이미지 대체 텍스트 제공으로
모든 사용자가 편리하게 이용할 수 있는 웹을
만드세요.

05

📝 릴리즈 노트 작성

프로젝트의 변경 사항을 기록하고 문서화하여 포트폴리오의 발전 과정을 명확하게 보여주고,
방문자에게 신뢰를 줄 수 있습니다.

Q&A GUIDE



문제를 마주했을 때, 정확한 질문은 빠른 해결의 열쇠입니다.
효율적인 도움을 받기 위해 다음 형식을 참고하여 질문해주세요.

-  **재현 절차:** 어떤 순서로 작업했을 때 문제가 발생했나요?
-  **에러 로그:** 터미널이나 콘솔의 에러 메시지 전문을 공유해주세요.
-  **스크린샷:** 화면 깨짐이나 UI 문제는 스크린샷이 필수입니다.
-  **최소 재현 코드:** 문제가 발생하는 부분만 간추린 코드를 보여주세요.



THANK

YOU.

오늘 수업이 여러분의 개발자 커리어에
성공적인 첫 걸음이 되기를 응원합니다.

NEXT STEPS

- ✓ 완성된 포트폴리오 URL 커뮤니티에 공유하기
- ✓ GitHub Repository 'About' 섹션에 태그 추가
- ✓ LinkedIn 프로필 'Projects'에 등록하기

PORTFOLIO URL

 github.io/aebonleee/portfolio

EMAIL

 aebon@hs.ac.kr

GITHUB

 [@vibe-coding](https://github.com/vibe-coding)

COMMUNITY

<https://padlet.com/aebon/vibe26>

문제 해결 Guide

자주 발생하는 3가지 핵심 에러와 해결 솔루션

Git 충돌 (Conflict)



팀원과 동일한 파일의 같은 라인을 수정하여 병합할 수 없는 상태입니다.
Rebase나 Merge로 수동 해결이 필요합니다.

SOLUTION

1. `git pull --rebase origin main`
2. 충돌파일 수동 수정 후 저장
3. `git add .` 후 `git rebase --continue`

배포 실패 / 404



GitHub Pages 배포 후 사이트가 열리지 않거나 404 페이지가 표시되는 경우입니다.

CHECKLIST

- Settings > Pages 브랜치 `main` / `root` 확인
- Actions 탭에서 빌드 에러 로그 확인
- 캐시 초기화: `Ctrl+Shift+R` (강력 새로고침)

CORS 정책 오류



클라이언트(브라우저)에서 외부 API 서버로 직접 요청 시 보안 정책으로 인해 차단되는 현상입니다.

WORKAROUND

- GitHub Pages는 정적 호스팅이므로 서버 설정 불가.
1. `CORS` 허용된 공개 `API` 사용
 2. 개발 시엔 로컬 프록시 설정 사용

PRACTICE CHECKPOINT

01

환경 설정 완료 확인

아래 항목을 모두 통과하면 다음 단계로 진행합니다.



01

VS Code 설치/실행 OK

Visual Studio Code가 정상적으로 설치되었고,
오류 없이 실행되는지 확인하세요.

02

버전 확인 (Git / Node)

터미널에서 `git --version`, `node -v` 명령어가
버전을 정상 출력해야 합니다.

03

확장 프로그램 설치

Live Server, Prettier, Auto Rename Tag
확장이 설치되었고 활성화 상태인지 점검하세요.

04

Git 사용자 설정

`git config`로 user.name과 user.email이
올바르게 등록되었는지 확인하세요.

05

Live Server 접속 확인

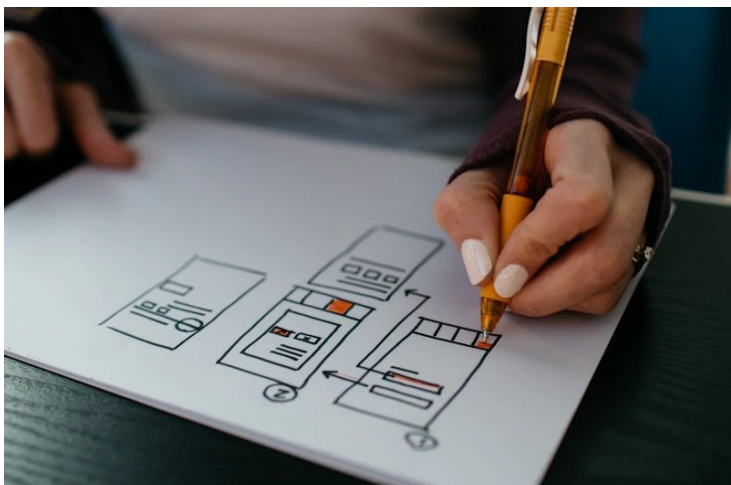
샘플 index.html 파일을 만들고 Go Live 버튼을 눌러 `http://127.0.0.1:5500/` 접속을 확인하세요.

PRACTICE

CHECK

POINT 02

포트폴리오 기본 완성 단계
핵심 점검 리스트입니다.



01

</> 시맨틱 HTML 구조

index.html 내부에 header, main, footer 등 의미론적 태그가 올바르게 배치되었는지 확인하세요.

02

🔗 뷰포트 및 CSS 연결

meta viewport 태그가 선언되어 있고, 외부 style.css 파일이 정상적으로 로드되는지 점검하세요.

03

👤 이미지 대체 텍스트

모든 img 태그에 alt 속성이 작성되어 웹 접근성 (Accessibility)을 준수하는지 확인하세요.

04

🌐 Live Server 렌더링

로컬 서버(127.0.0.1)를 통해 폰트와 레이아웃이 브라우저에서 의도한 대로 표시되는지 확인하세요.

05

📱 모바일 반응형 점검

개발자 도구의 디바이스 톨바를 사용하여 360~414px 해상도에서 레이아웃 깨짐이 없는지 반드시 테스트하세요.

DEPLOYMENT SUCCESS CHECKLIST

실습 체크포인트 3 :

GitHub Pages 배포 성공 여부를 확인하는
최종 점검 리스트입니다.



01

📶 Git Push 완료

git remote add, commit, push 명령어가 오류 메시지 없이 정상적으로 실행되었는지 터미널을 확인하세요.

02

CHECKPOINT 03

⚙️ Pages 설정 확인

GitHub Settings > Pages 메뉴에서 Branch가 'main'으로, 폴더가 '/(root)'로 올바르게 설정되었는지 점검하세요.

03

🌐 URL 접속 상태

제공된 배포 URL에 접속했을 때 HTTP 200 OK 상태로 첫 화면이 깨짐 없이 정상 로드되는지 확인하세요.

04

📱 반응형 동작 검증

PC 브라우저와 모바일 기기(또는 개발자 도구) 모두에서 레이아웃이 화면 폭에 맞춰 정상적으로 표시되어야 합니다.

05

✔️ 최종 에러 및 링크 검증

개발자 도구(F12) 콘솔에 빨간색 에러 메시지가 없고, 네비게이션과 버튼 등 페이지 내 모든 링크가 404 없이 정상 작동하는지 최종 확인하세요.

Prompt Optimization

AI 코딩 성능을 극대화하는 3가지 실전 전략



역할 + 목표 설정

단순한 코드 요청이 아닌, 전문가 페르소나와 달성하고자 하는 구체적인 품질 목표를 함께 제시합니다.

BEST PRACTICE

"프론트엔드 멘토로서, **접근성(WCAG)**을 **준수한 시맨틱 HTML**과 반응형 CSS 코드를 제안해줘."



맥락 + 제약 조건

현재 프로젝트의 폴더 구조, 스타일 가이드, 해상도 등 구체적인 제약 사항을 제공하여 재수정을 방지합니다.

BEST PRACTICE

"브레이크포인트는 **480/768/1024px**로 설정하고, 1280x720 픽셀 안전 영역을 준수해주세요."



명확한 출력 형식

결과물의 포맷을 지정하여(코드만, Diff, 체크리스트 등) 검증과 적용이 쉬운 형태로 답변을 받습니다.

BEST PRACTICE

"설명은 줄이고 **코드 Diff 포맷**과 수정 사유를 체크리스트로 정리해서 출력해줘."

EFFECTIVE DEBUGGING

실전 팁 - 디버깅 전략

AI DEBUGGING PROMPT

[에러 로그]

Uncaught TypeError: Cannot read properties of null...

[현재 상황]

페이지 로드 직후 버튼 클릭 시 발생합니다.

[요청 사항]

위 에러의 원인을 분석하고,
DOM 로딩 시점을 고려한 수정 코드를 제안해주세요.

PRE-CHECK LIST

1. 에러 재현 단계를 명확히 정리했나요?
2. 문제가 발생하는 최소한의 코드를 추출했나요?
3. `console.log`로 DOM 요소 존재 여부를 확인했나요?

ERROR ANALYSIS & FIX

Console & app.js

> Uncaught TypeError: Cannot read properties of null (reading 'addEventListener') at app.js:23

// ❌ 문제 원인: 스크립트가 DOM 렌더링 전에 실행됨

```
const btn = document.querySelector('#submit-btn');  
btn.addEventListener('click', handleSubmit); // btn is null!
```

// ✅ 해결 방안: DOMContentLoaded 이벤트 리스너 추가

```
document.addEventListener('DOMContentLoaded', () => {
```

// 안전하게 DOM 요소 선택

```
const btn = document.querySelector('#submit-btn');
```

```
  if (btn) {  
    btn.addEventListener('click', handleSubmit);  
    console.log('Event listener attached');  
  }  
});
```

PERFORMANCE MEASUREMENT

01 Lighthouse (Chrome DevTools)

Chrome 개발자 도구 내장 도구로 Performance, Accessibility, Best Practices, SEO 점수를 측정하며, 모든 항목 90점 이상 달성을 목표로 합니다.

02 PageSpeed Insights

구글의 공식 분석 도구를 활용하여 실제 사용자(Field) 데이터와 실험실(Lab) 데이터를 종합적으로 분석하고 모바일/데스크톱 최적화 가이드를 확인합니다.

03 핵심 최적화 전략

이미지 차세대 포맷(WebP/AVIF) 변환, CSS/JS 리소스 최소화 및 번들링, 렌더링 차단 리소스 제거를 통해 로딩 속도를 획기적으로 개선합니다.



FINAL LAUNCH CHECKLIST

배포 전 반드시 확인해야 할
5가지 핵심 점검 사항입니다.



01

성능 (Performance)

이미지 압축(WebP), Lazy-loading 적용, 불필요한 JS/CSS 제거 및 번들 최소화를 확인하세요.

02

SEO 최적화

title, meta description, OG 태그가 올바른지 확인하고, 시맨틱 태그 구조를 준수했는지 점검하세요.

03

웹 접근성 (A11y)

모든 이미지에 대체 텍스트(alt)가 있는지, 키보드만으로 내비게이션이 가능한지, 색상 대비가 충분한지 확인하세요.

04

보안 (Security)

HTTPS가 활성화되었는지, 민감한 API 키나 개인정보가 코드에 노출되지 않았는지(.env 사용) 확인하세요.

05

품질 및 호환성 (QA)

콘솔 에러가 0인지, 깨진 링크(404)가 없는지 확인하고, 다양한 디바이스와 브라우저에서 반응형 레이아웃이 깨지지 않는지 최종 점검하세요.

CLASS TIMELINE GUIDE



3시간 수업의 효율적인 진행을 위한 시간 배분 및 휴식 가이드입니다.

0-
40m

소개 및 환경 설정

강의 개요, VS Code 및 Git 설치, Claude/Gemini 계정 생성 및 API 키 발급을 완료합니다.

40-
80m

AI 도구 실습

Claude와 Gemini 기본 사용법 실습 후, 본격적인 코딩 전 10분간 휴식합니다.

10m BREAK

90-130m

포트폴리오 제작

HTML 구조 설계, CSS 스타일링, 반응형 구현 등 핵심 코딩을 집중적으로 진행합니다.

130-150m

GitHub 배포

Git 푸시 및 Pages 설정을 완료하고 배포 성공을 확인한 뒤 잠시 휴식합니다.

5m BREAK

180m

고급 기능 및 마무리

다크모드 등 추가 기능을 구현하고, Q&A 세션을 통해 학습 내용을 정리하며 수업을 마칩니다.

FAQ & Troubleshooting

바이브코딩 중 자주 마주하는 문제와 해결 가이드



코드 실행 에러

"AI가 짠 코드가 작동하지 않아요."
단순히 안 된다고 하기보다
구체적인 정보를 제공해야 합니다.

SOLUTION

"터미널의 **에러 메시지 전체**와 현재 작성된 관련
코드를 함께 복사해서 제공하세요."



배포 후 404 에러

GitHub Pages 배포 후 페이지를
찾을 수 없다면 파일 위치와 설정을
확인해야 합니다.

CHECKPOINT

"저장소 최상위에 **index.html**이 있는지 확인하고,
배포 브랜치가 main인지 점검하세요."



스타일 깨짐 현상

로컬에서는 잘 되는데 배포 후
이미지가 안 보이거나 스타일이 적용되지 않는
경우입니다.

CORRECTION

"절대 경로(/css) 대신 **상대 경로(./css)**를
사용하여 리소스를 연결하세요."

09

추가 학습 리소스 CURATED LINKS

RESOURCES

"배움은 끝이 없습니다.
공식 문서와 활발한 커뮤니티를 통해 개발 역량을 지속적으로 확장하세요."

📖 공식 문서

Claude API Docs
Gemini AI Studio
GitHub Docs
MDN Web Docs

</> 실전 튜토리얼

freeCodeCamp
Web.dev (Google)
CSS-Tricks

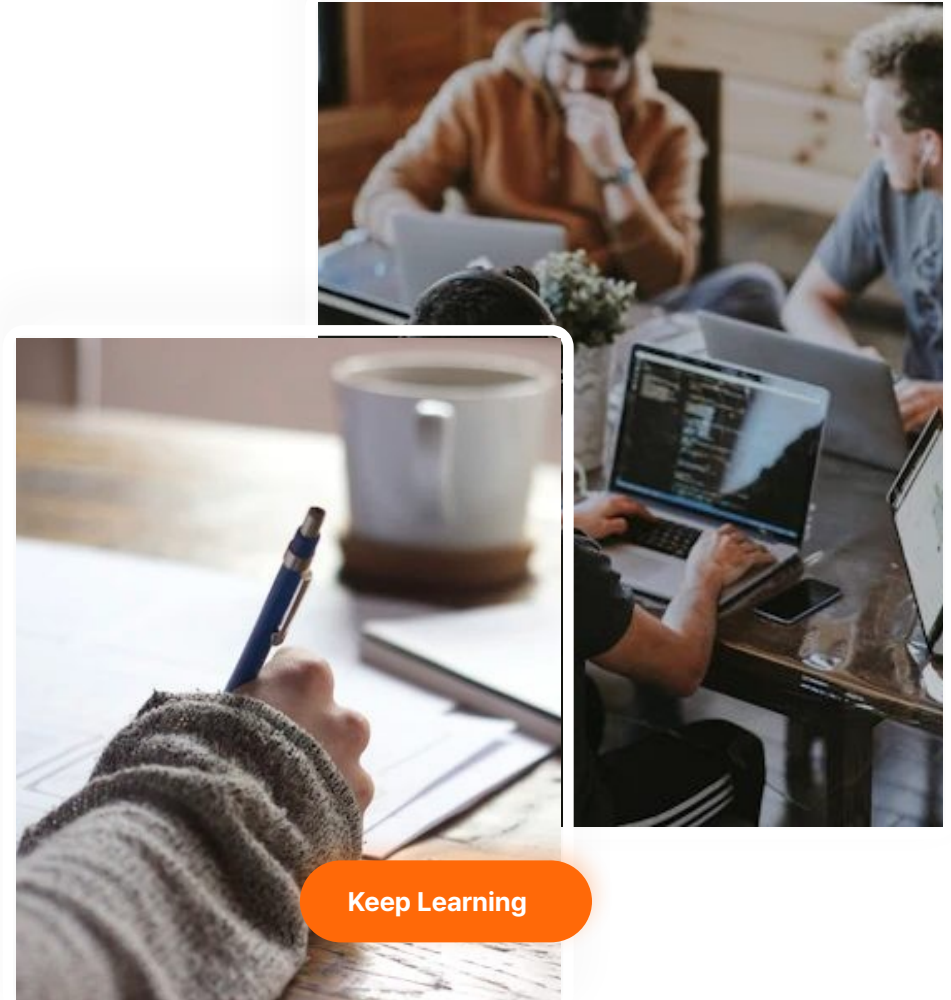
💡 포트폴리오 영감

Awwwards
Behance
Dribbble

👥 커뮤니티

Stack Overflow
GitHub Discussions
Reddit r/webdev

<https://padlet.com/aebon/vibe26>



CREDITS · CONTACT

출처 · 문의

이애본 (Aebon Lee, Ph.D) — 드림아이티비즈 대표 · 한신대학교 AI·SW대학 겸임교수

useful-ai.dreamitbiz.com — 강의안·영상 강의 라이브러리

aebon@dreamitbiz.com · 카카오톡 ID aebon

© 2026 이애본(드림아이티비즈). 교육 목적 비영리 이용 · 출처 표기